เอกสารประกอบการสอน

วิชา โครงสรางระบบคอมพิวเตอร์

(Computer System Organization)

บทที่ 3: ชุดคำสั่งและวัฏจักรคำสั่งของ CPU

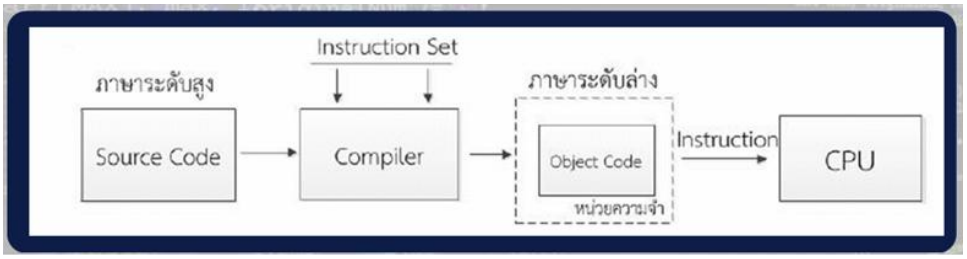
(Instruction Set and CPU Instruction Cycle)

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

1. บอกความหมายองค์ประกอบของชุดคำสั่ง (Instruction Set) ได้
2. อธิบายหน้าที่และการทำงานของ OpCode และ Operand ได้
3. อธิบายกระบวนการทำงานในวัฏจักรคำสั่งของ CPU (Instruction Cycle) ได้
4. อธิบายการทำงานชองกระบวนการ Instruction Address Calculation (IAC) ได้
5. อธิบายการทำงานชองกระบวนการ Instruction Fetch (IF) ได้
6. อธิบายการทำงานชองกระบวนการ Instruction Operation Decoding (IOD) ได้
7. อธิบายการทำงานชองกระบวนการ Execute (การปฏิบัติการ) ได้
8. อธิบายการทำงานของกระบวนการ OAC (Operand Address Calculation) ได้
9. อธิบายการทำงานของกระบวนการ OF (Operand Fetch) ได้
10. อธิบายการทำงานของกระบวนการ DO (Data Operation) ได้
11. อธิบายการทำงานของกระบวนการ OS (Operand Store) ได้
12. อธิบายส่วนประกอบคำสั่ง รหัสปฏิบัติการ (Operation Code) ได้
13. อธิบายส่วนประกอบคำสั่ง การอ้างถึงตัวถูกดำเนินการ (Operand Reference) ได้
14. อธิบายส่วนประกอบคำสั่ง การอ้างถึงคำสั่งถัดไป (Next Instruction Reference) ได้
15 อธิบายการออกแบบชุดคำสั่ง Operation Repertory (ชุดของการปฏิบัติการ) ได้
16 อธิบายการออกแบบชุดคำสั่ง Data Type (ชนิดข้อมูล) ได้
17 อธิบายการออกแบบชุดคำสั่ง Instruction Format (รูปแบบคำสั่ง) ได้
18 อธิบายการออกแบบชุดคำสั่ง Register (รีจิสเตอร์) ได้
19 อธิบายการออกแบบชุดคำสั่ง Addressing (การกำหนดแอดเดรส) ได้
20. อธิบายการทำงานประเภทของ Operation Data Transfer (คำสั่งย้ายข้อมูล) ได้
21. อธิบายการทำงานประเภทของ Operation Arithmetic (คำสั่งคำนวณทางคณิตศาสตร์) ได้
22. อธิบายการทำงานประเภทของ Operation Logical (คำสั่งตรรกะ) ได้
23. อธิบายการทำงานประเภทของ Operation System Control(คำสั่งควบคุมระบบ) ได้
24. อธิบายการทำงานประเภทของ Operation Input/output (คำสั่ง Input/Output) ได้
25. อธิบายการทำงานประเภทของ Operation Conversion (คำสั่งแปลงข้อมูล) ได้
26. จงอธิบาย Operation System Control คำสั่ง Unconditional Branch และ Conditional Branch ได้
27 จงอธิบาย Operation System Control คำสั่ง Skip การข้ามคำสั่งถัดไป หากเงื่อนไขที่กำหนดเป็นจริง ได้
28. จงอธิบาย Operation System Control ของคำสั่ง Procedure All ได้
29. จงอธิบาย Operation System Control ของคำสั่ง Subroutine Call ได้

30. จงอธิบาย Operation System Control ของคำสั่งควบคุมอุปกรณ์ Input/Output ได้
31. จงอธิบายการเขียนรหัสคำสั่งแบบ 4-Address Instruction (คำสั่งแบบ 4 ที่อยู่) ได้
32. จงอธิบายการเขียนรหัสคำสั่งแบบ 3-Address Instruction (คำสั่งแบบ 3 ที่อยู่) ได้
33. จงอธิบายการเขียนรหัสคำสั่งแบบ 2-Address Instruction (คำสั่งแบบ 2 ที่อยู่) ได้
34. จงอธิบายการเขียนรหัสคำสั่งแบบ 1 Address Instruction ไ(คำสั่งแบบ 1 ที่อยู่) ด้
35.จงอธิบายการทำงานคำสั่ง Immediate Addressing Mode (การกำหนดแอดเดรสแบบทันที) ได้
36.จงอธิบายการทำงานคำสั่ง Register Addressing Mode (การกำหนดแอดเดรสแบบรีจิสเตอร์) ได้
37.จงอธิบายการทำงานคำสั่ง Direct Addressing Mode (การกำหนดแอดเดรสแบบโดยตรง) ได้
38.จงอธิบายการทำงานคำสั่ง Register Indirect Addressing Mode (การกำหนดแอดเดรสแบบรีจิสเตอร์ทางอ้อม) ได้
39.จงอธิบายการทำงานคำสั่ง Indexed Addressing Mode (การกำหนดแอดเดรสแบบแทนที่/ดัชนี) ได้
40.จงอธิบายการทำงานคำสั่ง Relative Addressing Mode (การกำหนดแอดเดรสแบบสัมพันธ์) ได้
41.จงอธิบายการทำงานของคำสั่ง Stack Addressing Mode (การกำหนดแอดเดรสโดยใช้สแตก) ได้

1. ความหมายและองค์ประกอบของชุดคำสั่ง (Instruction Set)



รูปที่ 3-1 กระบวนการแปลงรหัสต้นฉบับ (Source Code) เป็นภาษาเครื่องเพื่อการประมวลผลโดย CPU

- **ความหมายของชุดคำสั่ง (Instruction Set):** คือกลุ่มของคำสั่งพื้นฐานทั้งหมดที่หน่วยประมวลผลกลาง (CPU) ของคอมพิวเตอร์นั้นๆ สามารถเข้าใจและปฏิบัติตามได้ เปรียบเสมือนภาษาเฉพาะที่ CPU ใช้ในการสื่อสารและควบคุมการทำงานของระบบ คำสั่งเหล่านี้จะบอก CPU ว่าต้องทำอะไรกับข้อมูลหรือที่ใด
- **องค์ประกอบของชุดคำสั่ง:** ชุดคำสั่งหนึ่งชุดจะประกอบด้วย Instruction ต่างๆ ที่ถูกออกแบบมาเพื่อทำหน้าที่เฉพาะเจาะจง เช่น คำสั่งสำหรับการบวก (ADD), ลบ (SUBTRACT), ย้ายข้อมูล (MOVE), เปรียบเทียบ (COMPARE) เป็นต้น

2. หน้าที่และการทำงานของ OpCode และ Operand

- **Operation Code (OpCode):** คือรหัสส่วนหนึ่งของคำสั่งที่บอก CPU ว่าจะต้องดำเนินการ "ปฏิบัติการ" (Operation) อะไรบ้าง เช่น OpCode สำหรับคำสั่ง "บวก" (ADD), "ลบ" (SUBTRACT), "โหลดข้อมูล" (LOAD) เป็นต้น OpCode เป็นส่วนสำคัญที่ทำให้ CPU รู้ว่าต้องทำอะไร
- **Operand:** คือส่วนประกอบของคำสั่งที่ระบุ "ข้อมูล" หรือ "ที่อยู่" (Address) ของข้อมูลที่ OpCode นำไปใช้ในการปฏิบัติการนั้นๆ หรือระบุตำแหน่งที่จะจัดเก็บผลลัพธ์ เช่น ถ้ามีคำสั่ง ADD A, B; OpCode คือ ADD ส่วน A และ B คือ Operand เป็นข้อมูลหรือที่อยู่ของข้อมูลที่จะถูกนำไปบวกกัน

โครงสร้างของคำสั่ง (Instruction Format) โดยทั่วไป:

สมมติว่าเรามีคำสั่งหนึ่งคำสั่ง (Instruction) ในหน่วยความจำ เมื่อ CPU ดึงคำสั่งนี้เข้ามาประมวลผล CPU จะมองว่าคำสั่งนี้ประกอบด้วยส่วนสำคัญ 2 ส่วนหลักๆ คือ:

```
+-----------------------------------+
|         Instruction (คำสั่ง)      |
+-----------------------------------+
|  OpCode  |      Operand(s)        |
| (รหัสปฏิบัติการ) | (ข้อมูล/ที่อยู่ข้อมูล) |
+-----------------------------------+
```

รูปที่ 3-2 ส่วนประกอบพื้นฐานของชุดคำสั่งคอมพิวเตอร์

ส่วนของ OpCode (Operation Code): จะอยู่ทางด้านซ้ายของคำสั่ง (หรือส่วนต้นของคำสั่ง) เป็นกลุ่มของบิตที่แทนรหัสเฉพาะสำหรับปฏิบัติการ เช่น

- ถ้า OpCode คือ 0001 อาจหมายถึงคำสั่ง ADD (บวก)
- ถ้า OpCode คือ 0010 อาจหมายถึงคำสั่ง SUB (ลบ)

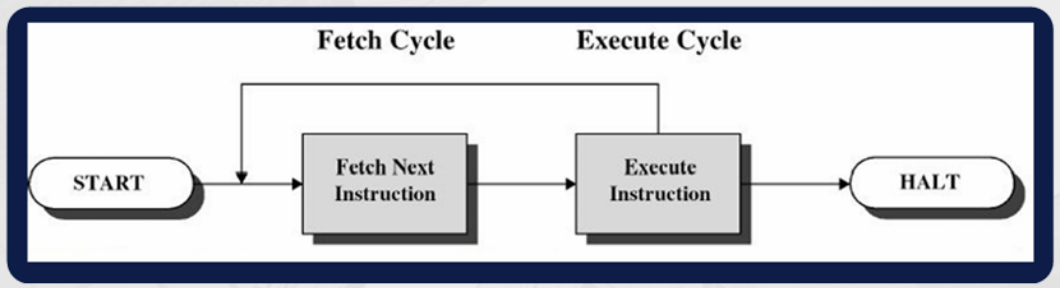
- ถ้า OpCode คือ 0011 อาจหมายถึงคำสั่ง LOAD (โหลดข้อมูล) CPU จะถอดรหัส (Decode) OpCode นี้เพื่อรู้ว่าต้องทำอะไร

- **ส่วนของ Operand(s):** จะอยู่ถัดจาก OpCode (หรือทางด้านขวา) เป็นส่วนที่ระบุว่าปฏิบัติการนั้นๆ จะกระทำกับอะไร โดยอาจเป็น:
  - **ข้อมูลโดยตรง (Immediate Value):** ตัวเลขที่เป็นค่าจริงๆ อยู่ในคำสั่งเลย
  - **ที่อยู่ในหน่วยความจำ (Memory Address):** ตำแหน่งของข้อมูลในหน่วยความจำ
  - **ชื่อรีจิสเตอร์ (Register Name):** ชื่อของรีจิสเตอร์ที่เก็บข้อมูล
  - **ที่อยู่ของอุปกรณ์ I/O:** พอร์ตหรือตำแหน่งของอุปกรณ์รับเข้า/ส่งออก

  ตัวอย่างการทำงาน: *สมมติ CPU ดึงคำสั่งที่มีรูปแบบ [OpCode] [Operand] เข้ามา:*
  - *ถ้า CPU ดึง [0001] [Address_A] เข้ามา:*
    - *0001 (OpCode) บอกว่าให้ทำปฏิบัติการ "บวก"*
    - *Address_A (Operand) บอกว่าให้ไปนำข้อมูลจาก "ที่อยู่ A" มาใช้ในการบวก (โดยอาจบวกกับค่าใน Accumulator หรือ Register อื่นๆ)*

3. กระบวนการทำงานในวัฏจักรคำสั่งของ CPU (Instruction Cycle)

- **วัฏจักรคำสั่ง (Instruction Cycle):** คือลำดับขั้นตอนที่ CPU ใช้ในการดึง (Fetch), ตีความ (Decode), และปฏิบัติ (Execute) คำสั่งจากหน่วยความจำอย่างต่อเนื่องเป็นวงจร การทำงานของโปรแกรมจะดำเนินไปตามวัฏจักรนี้จนกว่าคำสั่งจะระบุให้หยุดการทำงาน หรือมีการปิดเครื่อง



รูปที่ 3-3 ขั้นตอนการทำงานของวงจรคำสั่งพื้นฐาน (Basic Instruction Cycle)

การทำงานวัฏจักรคำสั่งของ CPU จะแสดงเป็นผังงาน (Flowchart) หรือไดอะแกรมวงจร ซึ่งแสดงการทำงานที่ต่อเนื่องและเป็นขั้นตอนของ CPU โดยทั่วไปจะเป็นวงจรวนซ้ำ (Loop) ที่ประกอบด้วย 2 ระยะหลักดังนี้:

1. **จุดเริ่มต้น (Start):** เมื่อคอมพิวเตอร์เริ่มทำงาน หรือโปรแกรมเริ่มต้น

2. **ระยะ FETCH (การดึงคำสั่ง):**

CPU จะมองหาตำแหน่งของคำสั่งถัดไปที่จะต้องประมวลผล (โดยใช้ค่าจาก Program Counter - PC) จากนั้น CPU จะทำการดึง (Fetch) คำสั่งนั้นจากหน่วยความจำหลัก (Main Memory) เข้ามาเก็บไว้ในรีจิสเตอร์ภายใน CPU (เช่น Instruction Register - IR)

3 **ระยะ EXECUTE (การปฏิบัติการคำสั่ง):**

การถอดรหัส (Decode): CPU จะทำการวิเคราะห์และถอดรหัสคำสั่งที่อยู่ใน IR เพื่อทำความเข้าใจว่าคำสั่งนั้นคืออะไร (OpCode) และต้องดำเนินการกับข้อมูลหรือที่ใด (Operand)

การดำเนินการ (Perform Operation): CPU จะดำเนินการตามปฏิบัติการที่ถอดรหัสได้ เช่น หากเป็นคำสั่งบวก ก็จะนำข้อมูลที่เกี่ยวข้องมาบวกกัน แล้วอาจเก็บผลลัพธ์ไว้ในรีจิสเตอร์หรือส่งกลับไปยังหน่วยความจำ

4. **การวนซ้ำ (Loop Back):**

เมื่อ CPU ทำการ Execute คำสั่งปัจจุบันเสร็จสิ้นแล้ว หากไม่มีคำสั่งให้หยุดทำงานหรือไม่มีการกระโดดไปยังคำสั่งอื่น (Branch/Jump) CPU จะวนกลับไปที่ **ระยะ FETCH** อีกครั้ง เพื่อดึงคำสั่งถัดไปมาประมวลผล โดย Program Counter (PC) จะถูกอัปเดตให้ชี้ไปยังคำสั่งถัดไปโดยอัตโนมัติ

5. **จุดสิ้นสุด (Halt/Stop):**

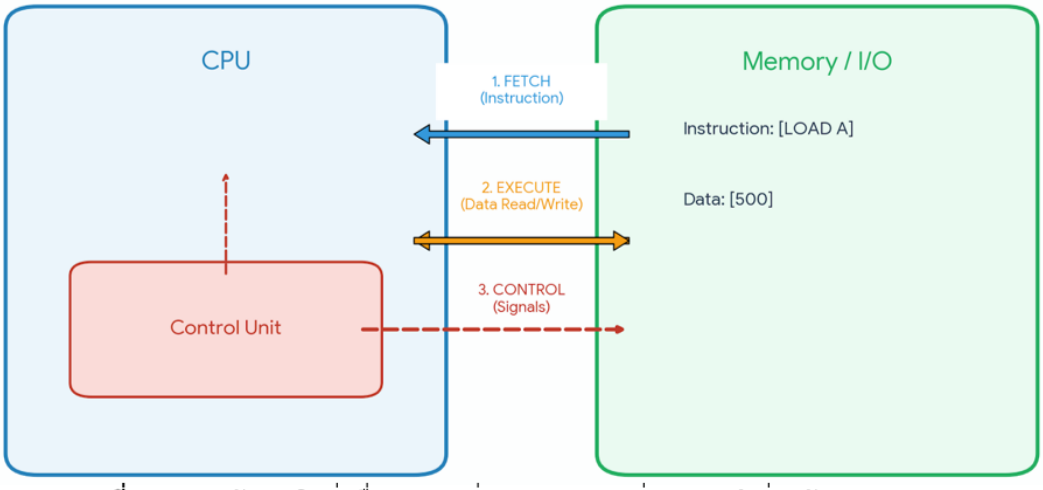
วัฏจักรนี้จะดำเนินไปเรื่อย ๆ จนกว่า CPU จะพบคำสั่งที่ระบุให้ "หยุดการทำงาน" (HALT Instruction) หรือเมื่อคอมพิวเตอร์ถูกปิด

หลังจากนั้นขั้นตอนหลักในการทำงาน: ประกอบด้วย 2 ระยะใหญ่ คือ ระยะ Fetch (การดึงคำสั่ง) และระยะ Execute (การปฏิบัติการคำสั่ง)

1. **คำสั่ง (Instruction):** จะถูกดึงจากหน่วยความจำไปยัง CPU (Fetch)

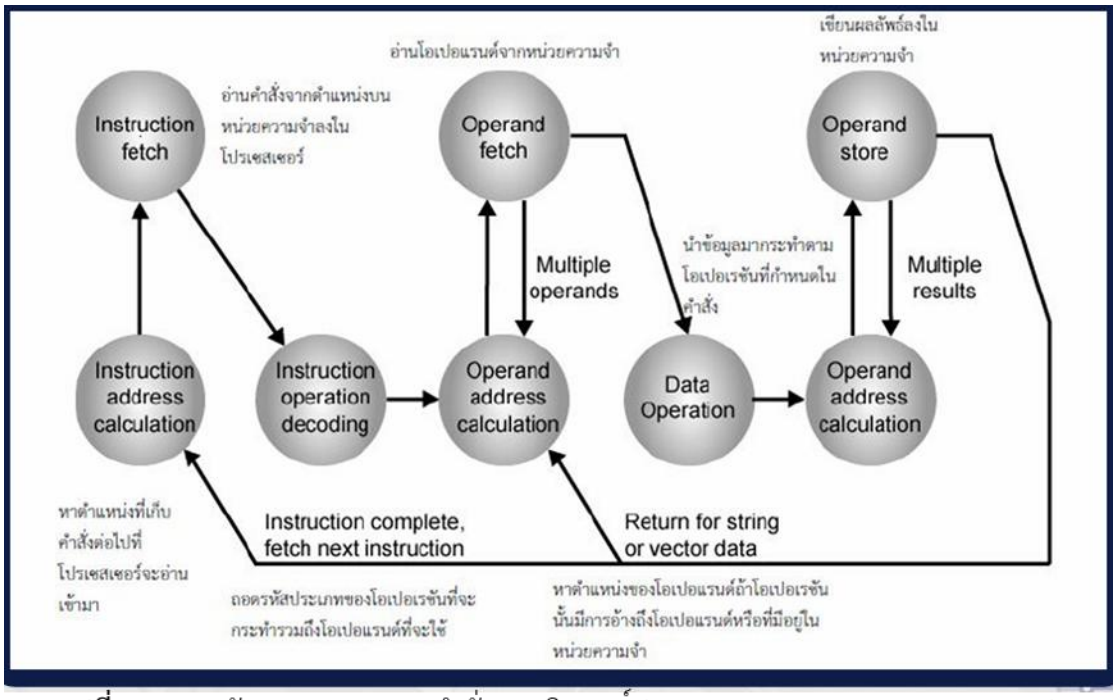
2. **ข้อมูล (Data):** อาจถูกดึงจากหน่วยความจำไปยัง CPU หรือจาก CPU ไปยังหน่วยความจำ/I/O ระหว่างระยะ Execute

3. **สัญญาณควบคุม (Control Signals):** CPU จะส่งสัญญาณควบคุมไปยังส่วนต่างๆ ของระบบเพื่อควบคุมการทำงานในแต่ละขั้นตอนของวัฏจักร



รูปที่ 3-4 แผนผังการติดต่อสื่อสารระหว่าง CPU และหน่วยความจำผ่านสัญญาณควบคุม

4. การทำงานของกระบวนการ IAC, IF, IOD และ Execute



รูปที่ 3-5 แผนผังวงจรสถานะของคำสั่งคอมพิวเตอร์ (Instruction Cycle State Diagram)

- **Instruction Address Calculation (IAC):** เป็นขั้นตอนแรกที่ CPU ทำการคำนวณหรือหาตำแหน่งของคำสั่งถัดไปที่ต้องดึงจากหน่วยความจำ โดยทั่วไปค่าตำแหน่งนี้จะอยู่ในรีจิสเตอร์ Program Counter (PC)

  ตัวอย่างการทำงาน_Instruction Address Calculation (IAC): *(การหาที่อยู่ของคำสั่ง)*

  ** **แนวคิด:** CPU ต้องรู้ว่าจะไปหยิบคำสั่งถัดไปที่ไหน*

  ** **การทำงาน:***

  ** CPU มีรีจิสเตอร์ตัวหนึ่งชื่อ **Program Counter (PC)** ซึ่งเก็บ "ที่อยู่" ของคำสั่งถัดไปที่จะต้องทำอยู่แล้ว*

  ** ในขั้นตอนนี้ CPU แค่เตรียมพร้อมที่จะใช้ค่าจาก PC นี้เพื่อไปดึงคำสั่ง*

- **Instruction Fetch (IF):** เป็นขั้นตอนที่ CPU อ่าน (ดึง) คำสั่งจากตำแหน่งที่คำนวณได้ในขั้นตอน IAC จากหน่วยความจำหลักมาเก็บไว้ในรีจิสเตอร์ภายใน CPU เช่น Instruction Register (IR)

  ตัวอย่างการทำงาน_Instruction Fetch (IF): *(การดึงคำสั่ง)*

  ** **แนวคิด:** CPU ไปหยิบคำสั่งมาจากชั้นหนังสือ (หน่วยความจำ)*

  ** **การทำงาน:** * CPU นำ "ที่อยู่" ของคำสั่งที่ได้จาก PC ไปใส่ในรีจิสเตอร์ **Memory Address Register (MAR)** (รีจิสเตอร์ที่ใช้ชี้ตำแหน่งในหน่วยความจำ)*

  ** จากนั้น **Control Unit (หน่วยควบคุม)** ของ CPU จะสั่งให้หน่วยความจำ "อ่าน" (Read) ข้อมูลจากตำแหน่งนั้น*

** หน่วยความจำจะส่ง "คำสั่ง" ที่อยู่ในตำแหน่งนั้นมาตาม **Data Bus** (เส้นทางส่งข้อมูล)*

** CPU รับคำสั่งนั้นเข้ามาเก็บไว้ในรีจิสเตอร์ **Memory Buffer Register (MBR)** (รีจิสเตอร์ที่ใช้พักข้อมูลที่รับส่งกับหน่วยความจำ)*

** คำสั่งใน MBR จะถูกย้ายไปเก็บไว้ใน **Instruction Register (IR)** (รีจิสเตอร์ที่ใช้เก็บคำสั่งที่กำลังจะประมวลผล)*

** (ในขณะเดียวกัน) ค่าใน PC จะถูกเพิ่มขึ้นอัตโนมัติ เพื่อชี้ไปยังคำสั่งถัดไปสำหรับรอบต่อไป (เตรียมพร้อมสำหรับ IAC ในรอบหน้า)*

- **Instruction Operation Decoding (IOD):** เป็นขั้นตอนที่ CPU วิเคราะห์และตีความคำสั่งที่ถูกดึงมา (ที่อยู่ใน IR) เพื่อแยกแยะ OpCode และ Operand ออกจากกัน และทำความเข้าใจว่าคำสั่งนั้นต้องการให้ทำปฏิบัติการอะไร และต้องใช้ข้อมูลจากที่ใด

  ***ตัวอย่างการทำงาน_Instruction Operation Decoding (IOD):** (การตีความคำสั่ง)*

  ** **แนวคิด:** CPU อ่านคำสั่งที่หยิบมาได้ แล้วทำความเข้าใจว่า "ให้ทำอะไร" และ "กับอะไร"*

  ** **การทำงาน:***

  ** **Control Unit** จะรับคำสั่งที่อยู่ใน **IR** มาวิเคราะห์*

  ** **Control Unit** จะแยกส่วน **OpCode** (รหัสปฏิบัติการ เช่น บวก, ลบ, ย้ายข้อมูล) ออกจากส่วน **Operand** (ข้อมูล, ที่อยู่ข้อมูล, หรือชื่อรีจิสเตอร์ที่จะใช้ในปฏิบัติการ)*

  ** จากการวิเคราะห์นี้ CPU จะรู้ว่าคำสั่งนี้ต้องการให้ทำอะไร (เช่น บวกข้อมูล) และจะไปหาข้อมูลที่ใช้บวกนั้นได้จากที่ไหน*

- **Execute (การปฏิบัติการ):** เป็นขั้นตอนที่ CPU ดำเนินการตามปฏิบัติการที่ตีความได้จากคำสั่งนั้นๆ เช่น หากเป็นคำสั่ง ADD ก็จะทำการบวกข้อมูลที่ระบุใน Operand และเก็บผลลัพธ์ การปฏิบัติการอาจเกี่ยวข้องกับการเข้าถึงข้อมูลในรีจิสเตอร์ หน่วยความจำ หรือ I/O

  ***ตัวอย่างการทำงาน_Execute (การปฏิบัติการ):** (การลงมือทำตามคำสั่ง)*

  ** **แนวคิด:** CPU ลงมือทำตามสิ่งที่ตีความได้จากคำสั่ง*

  ** **การทำงาน:***

  ** **Control Unit** จะส่งสัญญาณควบคุมไปยังส่วนต่างๆ ของ CPU เพื่อให้ดำเนินการตามคำสั่งนั้นๆ*

  ** หากคำสั่งนั้นเป็น **คำสั่งทางคณิตศาสตร์หรือตรรกะ (Arithmetic/Logical Operation):***

  ** ข้อมูล Operand (หากไม่ได้อยู่ในรีจิสเตอร์ ก็จะดึงจากหน่วยความจำมา) จะถูกส่งไปยัง **ALU (Arithmetic Logic Unit)***

  ** ALU จะทำการบวก ลบ คูณ หาร หรือดำเนินการทางตรรกะตาม OpCode*

  ** ผลลัพธ์จะถูกเก็บไว้ในรีจิสเตอร์ (เช่น Accumulator) หรือเขียนกลับไปยังหน่วยความจำ*

** หากคำสั่งนั้นเป็น **คำสั่งย้ายข้อมูล (Data Transfer):***

** ข้อมูลจะถูกย้ายจากแหล่งหนึ่ง (รีจิสเตอร์, หน่วยความจำ, I/O) ไปยังปลายทาง (รีจิสเตอร์, หน่วยความจำ, I/O)*

** หากคำสั่งนั้นเป็น **คำสั่งควบคุม (Control Operation - เช่น Branch/Jump):***

** ค่าใน Program Counter (PC) อาจถูกเปลี่ยนแปลง เพื่อให้ชี้ไปยังคำสั่งอื่นที่ไม่ใช่คำสั่งถัดไปในลำดับปกติ*

หลังจากนั้นวงจร (Cycle): เมื่อขั้นตอน Execute เสร็จสิ้น CPU ก็จะวนกลับไปเริ่มต้นที่ Instruction Address Calculation (IAC) อีกครั้งเพื่อหาและดึงคำสั่งถัดไปมาประมวลผล ทำแบบนี้ไปเรื่อยๆ จนกว่าโปรแกรมจะจบการทำงาน

5. การทำงานของกระบวนการ OAC, OF, DO และ OS

หลังจากที่ CPU ได้ทำการดึง (Fetch) และตีความ (Decode) คำสั่งแล้ว CPU จะเข้าสู่ระยะ **Execute** เพื่อลงมือทำตามคำสั่งนั้น ซึ่งในระยะ Execute นี้เองที่อาจแบ่งย่อยออกเป็นขั้นตอนต่อไปนี้

OAC (Operand Address Calculation): การคำนวณที่อยู่ของตัวถูกดำเนินการ

แนวคิด: เมื่อ CPU รู้แล้วว่าคำสั่งต้องการทำอะไร (จาก OpCode) และต้องการข้อมูลตัวไหน (จาก Operand) แต่บางครั้ง Operand นั้นไม่ได้เป็นข้อมูลโดยตรง หรือไม่ได้เป็นชื่อรีจิสเตอร์ที่เก็บข้อมูล แต่เป็น "ที่อยู่" หรือ "วิธีการ" ที่จะไปหาที่อยู่ของข้อมูลนั้น (เช่น การกำหนดแอดเดรสแบบทางอ้อม, แบบดัชนี, แบบสัมพัทธ์)

การทำงาน: ในขั้นตอนนี้ CPU (โดยหน่วยควบคุม) จะทำการคำนวณหา "ที่อยู่จริง" (Effective Address) ของข้อมูลที่จะถูกนำมาใช้ในการปฏิบัติการ โดยใช้เทคนิคการกำหนดแอดเดรส (Addressing Mode) ที่ระบุในคำสั่ง เช่น:

- ถ้าเป็น Register Indirect Addressing: CPU จะอ่านค่าจากรีจิสเตอร์ที่ระบุ แล้วค่านั้นคือที่อยู่ของข้อมูล
- ถ้าเป็น Indexed Addressing: CPU นำค่าจากรีจิสเตอร์ดัชนีไปบวกกับค่าคงที่ในคำสั่ง เพื่อหาที่อยู่จริง

- **เป้าหมาย:** เพื่อให้ได้ที่อยู่หน่วยความจำที่แน่นอนของข้อมูลที่ต้องการ

OF (Operand Fetch): การดึงตัวถูกดำเนินการ

แนวคิด: เมื่อรู้ที่อยู่ของข้อมูลแล้ว ก็ต้องไป "หยิบ" ข้อมูลนั้นมา

การทำงาน: CPU จะนำ "ที่อยู่จริง" ที่คำนวณได้จากขั้นตอน OAC ไปใช้ในการอ่านข้อมูล (Operand) จากหน่วยความจำหลัก

- คล้ายกับขั้นตอน Instruction Fetch แต่คราวนี้ดึง "ข้อมูล" ไม่ใช่ "คำสั่ง"

- ข้อมูลที่ได้จะถูกนำมาเก็บไว้ในรีจิสเตอร์ภายใน CPU ที่พร้อมจะนำไปใช้ในการประมวลผล (เช่น ในรีจิสเตอร์ทั่วไป หรือรีจิสเตอร์สำหรับ ALU)

เป้าหมาย: เพื่อให้ CPU มีข้อมูลพร้อมสำหรับการปฏิบัติการ

DO (Data Operation): การปฏิบัติการกับข้อมูล

แนวคิด: ขั้นตอนที่ CPU ลงมือ "ทำ" สิ่งที่คำสั่งต้องการจริงๆ

การทำงาน: ในขั้นตอนนี้ **ALU (Arithmetic Logic Unit)** ของ CPU จะถูกเรียกใช้งาน

- ถ้าเป็นคำสั่ง ADD, SUBTRACT: ALU จะทำการบวกหรือลบข้อมูลที่ดึงมา (จาก OF)
- ถ้าเป็นคำสั่ง AND, OR: ALU จะทำการปฏิบัติการทางตรรกะ
- ถ้าเป็นคำสั่ง SHIFT, ROTATE: ALU หรือส่วนควบคุมจะทำการเลื่อนบิตหรือหมุนบิตของข้อมูล
- หากเป็นคำสั่งควบคุม (เช่น Branch หรือ Jump) อาจมีการปรับเปลี่ยนค่าใน Program Counter (PC) โดยตรงในขั้นตอนนี้

เป้าหมาย: สร้างผลลัพธ์จากการประมวลผลตาม OpCode

OS (Operand Store): การจัดเก็บตัวถูกดำเนินการ (ผลลัพธ์)

แนวคิด: หลังจากได้ผลลัพธ์จากการปฏิบัติการแล้ว ต้องนำผลลัพธ์นั้นไปเก็บไว้ในที่ที่ถูกต้อง

การทำงาน: ผลลัพธ์ที่ได้จากขั้นตอน DO จะถูกเขียนกลับไปยังตำแหน่งที่กำหนดไว้

- อาจเป็นการเขียนกลับไปในรีจิสเตอร์ภายใน CPU
- อาจเป็นการเขียนกลับไปยังหน่วยความจำหลัก (โดยใช้ Address Bus และ Data Bus)
- อาจเป็นการส่งออกไปยังอุปกรณ์ I/O

เป้าหมาย: ทำให้ผลลัพธ์จากการประมวลผลสามารถนำไปใช้งานต่อได้ หรือบันทึกไว้

ตัวอย่างอธิบายลักษณะการทำงานที่เป็นภาพรวม

เมื่อตัว CPU กำลังทำภารกิจที่ได้รับมอบหมาย:

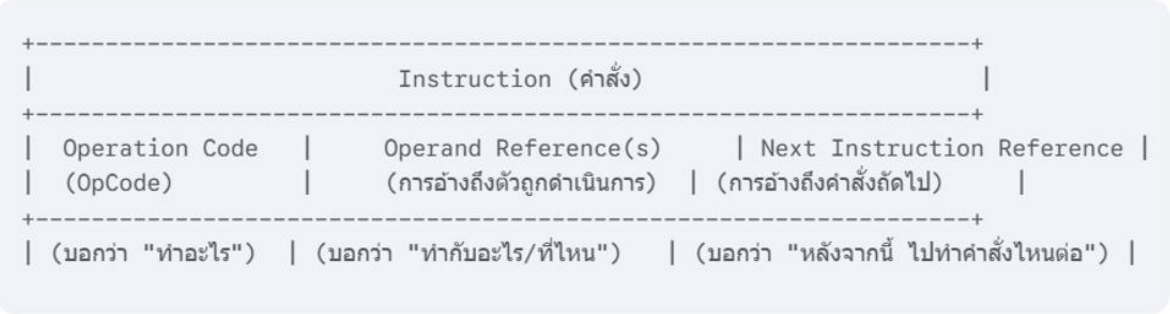
1. ***IAC:*** *"ฉันจะหาคำสั่งต่อไปจากที่ไหนนะ? (ดู PC)"*
2. ***IF:*** *"โอเค! เจอที่อยู่แล้ว ไปหยิบคำสั่งนั้นมาใส่กระเป๋า (IR) ก่อน"*
3. ***IOD:*** *"มาดูกันสิว่าคำสั่งนี้บอกให้ทำอะไร (OpCode) แล้วต้องทำงานกับข้อมูลตัวไหน (Operand)"*
4. ***OAC (ถ้าจำเป็น):*** *"อืม...ข้อมูลที่ต้องใช้ไม่ได้อยู่ตรงหน้า แต่บอกวิธีหามาให้ งั้นฉันต้องไปคำนวณหาที่อยู่จริงของมันก่อนนะ"*
5. ***OF (ถ้าจำเป็น):*** *"โอเค! ได้ที่อยู่จริงของข้อมูลแล้ว ไปหยิบข้อมูลนั้นมาใส่กระเป๋า (รีจิสเตอร์) พร้อมใช้"*
6. ***DO:*** *"ข้อมูลพร้อม! ลงมือทำตามคำสั่งเลย (ให้ ALU ทำงาน)"*

7. ***OS (ถ้าจำเป็น):*** *"งานเสร็จแล้ว! ผลลัพธ์ที่ได้จะเอาไปเก็บไว้ที่ไหนดีนะ? (เก็บในรีจิสเตอร์ หรือส่งกลับหน่วยความจำ)"*
8. *จากนั้น CPU ก็จะวนกลับไปที่ IAC อีกครั้ง เพื่อเริ่มภารกิจถัดไป*

6. การทำงานของส่วนประกอบคำสั่ง Operation Code, Operand Reference และ Next Instruction Reference

ส่วนประกอบคำสั่ง (Instruction Components) อธิบายได้ดังนี้

การทำงานของคำสั่งหนึ่งคำสั่งเป็นกล่องข้อมูลที่ CPU อ่านเข้ามา โดยในกล่องนั้นจะแบ่งออกเป็นช่องๆ ที่มีข้อมูลสำคัญแตกต่างกันไป



รูปที่ 3-6 โครงสร้างคำสั่งคอมพิวเตอร์แบบระบุตำแหน่งคำสั่งถัดไป

<u>ส่วนประกอบคำสั่ง Operation Code (OpCode): รหัสปฏิบัติการ</u>

- **หน้าที่:** เป็นส่วนที่สำคัญที่สุด เพราะเป็นตัวกำหนดว่า CPU จะต้อง "ทำอะไร" ครับ เปรียบเสมือนกริยาในประโยค
- **การทำงาน:** CPU จะตีความ OpCode นี้เพื่อรู้ว่าต้องดำเนินการทางคณิตศาสตร์ (เช่น บวก ลบ), ทางตรรกะ (เช่น AND OR), การย้ายข้อมูล (เช่น LOAD STORE), หรือการควบคุมการทำงาน (เช่น JUMP CALL)

  ตัวอย่าง: *ถ้า OpCode คือ 0001 อาจหมายถึง ADD (บวก)*
  ถ้า OpCode คือ 0010 อาจหมายถึง SUB (ลบ)

<u>ส่วนประกอบคำสั่ง Operand Reference(s): การอ้างถึงตัวถูกดำเนินการ</u>

- **หน้าที่:** เป็นส่วนที่บอกว่าปฏิบัติการที่กำหนดโดย OpCode นั้น จะต้อง "ทำกับข้อมูลอะไร" หรือ "ข้อมูลนั้นอยู่ที่ไหน" ครับ เปรียบเสมือนกรรมหรือส่วนขยายในประโยค
- **การทำงาน:** ส่วนนี้ไม่ได้บรรจุข้อมูลจริง ๆ เสมอไป แต่อาจเป็น:
  - **ข้อมูลโดยตรง (Immediate Value):** ตัวเลขที่ใช้ทันที บรรจุอยู่ในคำสั่งเลย เช่น ADD 5 (บวก 5 เข้าไป)
  - **ที่อยู่ในหน่วยความจำ (Memory Address):** ตำแหน่งของข้อมูลใน RAM เช่น LOAD 1000 (โหลดข้อมูลจากตำแหน่ง 1000)

- **ชื่อรีจิสเตอร์ (Register Name):** ชี้ไปยังรีจิสเตอร์ใน CPU ที่เก็บข้อมูลอยู่ เช่น ADD R1, R2 (บวกค่าในรีจิสเตอร์ R2 ไปยัง R1)
- **ตัวเลือกของ Addressing Mode:** ชี้บอกว่าต้องใช้วิธีใดในการหาข้อมูลที่แท้จริง (เช่น ทางอ้อม, ดัชนี, สัมพัทธ์)

- **ความสำคัญ:** ทำให้ CPU รู้ว่าต้องไปหยิบข้อมูลจากที่ไหน หรือผลลัพธ์จะเก็บไว้ที่ไหน

ส่วนประกอบคำสั่ง Next Instruction Reference: การอ้างถึงคำสั่งถัดไป

- **หน้าที่:** เป็นส่วนที่บอก CPU ว่า "หลังจากประมวลผลคำสั่งปัจจุบันเสร็จแล้ว จะไปหาคำสั่งถัดไปจากที่ไหน" ครับ
- **การทำงาน:** โดยทั่วไปแล้ว คอมพิวเตอร์ส่วนใหญ่จะใช้ **Program Counter (PC)** เป็นตัวชี้ไปยังที่อยู่ของคำสั่งถัดไปในลำดับที่ต่อเนื่องกัน (PC จะเพิ่มค่าขึ้นเอง) ดังนั้น ในคำสั่งส่วนใหญ่จึงไม่จำเป็นต้องมีส่วนนี้โดยตรง
  - **กรณีปกติ:** CPU จะเพิ่มค่า PC ขึ้นโดยอัตโนมัติเมื่อทำคำสั่งปัจจุบันเสร็จสิ้น
  - **กรณีคำสั่งควบคุม (Control Instructions):** อย่างไรก็ตาม สำหรับคำสั่งควบคุมการทำงาน เช่น **Branch (กระโดด)** หรือ **Jump (ข้าม)** หรือ **Call (เรียกใช้ซับรูทีน)** ส่วนนี้ของคำสั่งจะสำคัญมาก เพราะมันจะระบุ "ที่อยู่ใหม่" ที่ CPU ต้องกระโดดไปปฏิบัติคำสั่งต่อ ซึ่งจะทำให้ PC เปลี่ยนค่าไปตามที่อยู่ใหม่นี้

- **ความสำคัญ:** กำหนดทิศทางการไหลของโปรแกรม

7. การออกแบบการทำงานชุดคำสั่ง Operation Repertory, Data Type, Instruction Format, Register และ Addressing

- **Operation Repertory (ชุดของการปฏิบัติการ):** คือการกำหนดว่า CPU นั้นๆ จะสามารถทำปฏิบัติการอะไรได้บ้าง (เช่น บวก, ลบ, คูณ, หาร, ย้ายข้อมูล, ตรรกะ, ควบคุม) การออกแบบจะพิจารณาถึงความถี่ในการใช้งานและความซับซ้อนของแต่ละปฏิบัติการ

  ตัวอย่างเช่น Operation Repertory (ชุดของการปฏิบัติการ): "CPU นี้ทำอะไรได้บ้าง?"

  แนวคิด: เปรียบเหมือน "รายการความสามารถ" ของ CPU

  การออกแบบ: ผู้ผลิต CPU ต้องตัดสินใจว่า CPU จะรองรับการทำงานพื้นฐานอะไรบ้าง (เช่น บวก, ลบ, โหลดข้อมูล, กระโดด) ไปจนถึงการทำงานที่ซับซ้อนขึ้น (เช่น การคำนวณทศนิยม, การควบคุม I/O พิเศษ)

  สิ่งที่พิจารณา:

  ความถี่ในการใช้งาน: คำสั่งไหนที่โปรแกรมส่วนใหญ่ใช้บ่อยๆ ควรทำให้เร็วและง่ายต่อการใช้งาน

ความซับซ้อน: จะมีคำสั่งที่ซับซ้อนมาก (CISC) หรือคำสั่งพื้นฐานจำนวนมากแต่ทำได้เร็ว (RISC)

ตัวอย่าง: *อาจเป็นรายการคำสั่งเรียงเป็นแถว เช่น ADD, SUB, LOAD, STORE, JUMP, CALL โดยแต่ละคำสั่งมีคำอธิบายสั้นๆ กำกับ*

- **Data Type (ชนิดข้อมูล):** การออกแบบชุดคำสั่งต้องพิจารณาว่า CPU จะรองรับข้อมูลประเภทใดบ้าง เช่น ตัวเลขจำนวนเต็ม (Integer), ตัวเลขทศนิยม (Floating-Point), ตัวอักขระ (Character) หรือข้อมูลแบบตรรกะ (Boolean)

  ตัวอย่างเช่น Data Type (ชนิดข้อมูล): "CPU นี้เข้าใจข้อมูลแบบไหน?"

  - **แนวคิด:** เปรียบเหมือน "รูปแบบของวัตถุดิบ" ที่ CPU สามารถแปรรูปได้
  - **การออกแบบ:** กำหนดว่าข้อมูลที่จะถูกประมวลผลจะเป็นชนิดใด เช่น
    - **จำนวนเต็ม (Integer):** เลขจำนวนเต็มบวก/ลบ
    - **จำนวนจริง/ทศนิยม (Floating-Point):** เลขที่มีจุดทศนิยม
    - **ตัวอักขระ (Character):** เช่น ASCII หรือ Unicode
    - **ข้อมูลแบบตรรกะ (Boolean):** จริง/เท็จ (0/1)
  - **สิ่งที่พิจารณา:** ความต้องการของโปรแกรมและแอปพลิเคชันส่วนใหญ่ที่ต้องประมวลผลข้อมูลชนิดใดบ้าง
  - ***ตัวอย่าง:*** *อาจเป็นตารางที่แสดงชนิดข้อมูลต่างๆ และรูปแบบการจัดเก็บในหน่วยความจำ (เช่น Integer = 32 bits, Float = IEEE 754 format)*

- **Instruction Format (รูปแบบคำสั่ง):** การกำหนดโครงสร้างของคำสั่งว่าจะประกอบด้วยกี่บิต แต่ละส่วน (OpCode, Operand) มีขนาดเท่าใด และจัดเรียงอย่างไร เพื่อประสิทธิภาพในการดึงและตีความคำสั่ง

  ตัวอย่างเช่น Instruction Format (รูปแบบคำสั่ง): "โครงสร้างของคำสั่งแต่ละตัวเป็นอย่างไร?"

  - **แนวคิด:** เปรียบเหมือน "แบบฟอร์มมาตรฐาน" ที่ทุกคำสั่งต้องกรอกข้อมูลให้ตรงตามนั้น
  - **การออกแบบ:** กำหนดว่าคำสั่งหนึ่งคำสั่งจะมีขนาดกี่บิต (เช่น 16 บิต, 32 บิต, 64 บิต) และภายในคำสั่งนั้นจะแบ่งออกเป็นส่วนๆ อย่างไรบ้าง เช่น:
    - **OpCode Field:** ส่วนสำหรับรหัสปฏิบัติการ
    - **Operand Field(s):** ส่วนสำหรับระบุ Operand หรือที่อยู่ของ Operand
    - (อาจมี) **Next Instruction Address Field:** ส่วนสำหรับที่อยู่ของคำสั่งถัดไป (ในบางสถาปัตยกรรม)

- **สิ่งที่พิจารณา:** ความสมดุลระหว่างจำนวนปฏิบัติการที่รองรับ, จำนวน Operand ที่อ้างอิงได้, และขนาดโดยรวมของคำสั่ง
- ***ตัวอย่าง:*** *ตารางที่แบ่งออกเป็นช่องๆ โดยมีฉลากกำกับแต่ละช่องว่า OpCode, Operand 1, Operand 2 เป็นต้น*

Register (รีจิสเตอร์): การออกแบบคำสั่งต้องพิจารณาถึงจำนวนและประเภทของรีจิสเตอร์ภายใน CPU เนื่องจากรีจิสเตอร์เป็นหน่วยความจำที่เร็วที่สุดที่ CPU ใช้ในการเก็บข้อมูลชั่วคราวและ Operand ตัวอย่างเช่น **Register (รีจิสเตอร์): "CPU มีช่องเก็บข้อมูลด่วนกี่ช่องและใช้ทำอะไรบ้าง?"**

- **แนวคิด:** เปรียบเหมือน "โต๊ะทำงานพร้อมลิ้นชักด่วน" ภายใน CPU
- **การออกแบบ:** กำหนดจำนวนรีจิสเตอร์ (เช่น 8, 16, 32, 64 ตัว), ขนาดของแต่ละรีจิสเตอร์, และหน้าที่เฉพาะของรีจิสเตอร์บางตัว (เช่น Program Counter, Stack Pointer, General-Purpose Registers)
- **สิ่งที่พิจารณา:**
  - **ประสิทธิภาพ:** การใช้รีจิสเตอร์ช่วยลดการเข้าถึงหน่วยความจำที่ช้ากว่า
  - **ความซับซ้อนของคำสั่ง:** การมีรีจิสเตอร์มากช่วยให้คำสั่งสั้นลงได้ (เช่น 3-Address Instruction)
- ***ตัวอย่าง:*** *การทำงานของ CPU ที่มีกลุ่มของที่เก็บข้อมูลเล็กๆ อยู่ภายใน แสดงถึงรีจิสเตอร์แต่ละตัวพร้อมชื่อกำกับ (R0, R1, PC, SP)*

Addressing (การกำหนดแอดเดรส): การออกแบบวิธีการที่ CPU ใช้ในการระบุตำแหน่งของ Operand หรือผลลัพธ์ ซึ่งมีหลายรูปแบบ (Addressing Modes)

ตัวอย่างเช่น **Addressing (การกำหนดแอดเดรส): "CPU จะไปหาข้อมูลในหน่วยความจำได้อย่างไรบ้าง?"**

- **แนวคิด:** เปรียบเหมือน "วิธีการระบุที่อยู่" ของสิ่งของในโกดัง (หน่วยความจำ)
- **การออกแบบ:** กำหนดว่า CPU จะใช้วิธีการใดบ้างในการคำนวณหรือระบุตำแหน่งของ Operand ในหน่วยความจำ ซึ่งมีหลายรูปแบบ (Addressing Modes) เช่น:
  - **Immediate:** ข้อมูลอยู่ในคำสั่งโดยตรง
  - **Register:** ข้อมูลอยู่ในรีจิสเตอร์
  - **Direct:** คำสั่งบอกที่อยู่หน่วยความจำโดยตรง
  - **Indirect:** คำสั่งบอกรีจิสเตอร์ที่เก็บที่อยู่ของข้อมูล
  - **Indexed:** เอาค่าในรีจิสเตอร์มาบวกกับค่าคงที่เพื่อหาที่อยู่
  - **Relative:** เอาค่าใน Program Counter มาบวกกับค่าคงที่เพื่อหาที่อยู่

    - **Stack:** ข้อมูลอยู่บน Stack (ใช้ Stack Pointer)
- **สิ่งที่พิจารณา:** ความยืดหยุ่นในการเขียนโปรแกรม, ประสิทธิภาพในการเข้าถึงข้อมูล, และความซับซ้อนของวงจรภายใน CPU
- ***ตัวอย่าง:*** *อาจเป็นผังงานเล็กๆ ที่แสดงเส้นทางการเข้าถึงข้อมูลจากคำสั่งไปยังรีจิสเตอร์หรือหน่วยความจำในแต่ละโหมด*

8. การทำงานประเภทของ Operation Data Transfer, Operation Arithmetic และ Operation Logical

- **Operation Data Transfer (คำสั่งย้ายข้อมูล):** ทำหน้าที่ย้ายข้อมูลระหว่างส่วนต่างๆ ในระบบคอมพิวเตอร์ เช่น
    - **LOAD:** ย้ายข้อมูลจากหน่วยความจำไปยังรีจิสเตอร์
    - **STORE:** ย้ายข้อมูลจากรีจิสเตอร์ไปยังหน่วยความจำ
    - **MOVE:** ย้ายข้อมูลระหว่างรีจิสเตอร์
    - **IN/OUT:** ย้ายข้อมูลระหว่างรีจิสเตอร์กับอุปกรณ์ I/O

ตัวอย่างเช่น Operation Data Transfer (คำสั่งย้ายข้อมูล)

- **แนวคิด:** เปรียบเหมือน "การขนส่งข้อมูล" จากจุดหนึ่งไปยังอีกจุดหนึ่ง โดยไม่ได้เปลี่ยนแปลงตัวข้อมูลเลย
- **การทำงาน:**
    - **LOAD:** สมมติว่า "ข้อมูล" อยู่ใน "กล่อง (Memory Location)" ใน "โกดัง (Main Memory)" คำสั่ง LOAD ก็เหมือนการ "หยิบ" ข้อมูลจากกล่องนั้น มาวางไว้บน "โต๊ะทำงาน (Register)" ของ CPU เพื่อเตรียมพร้อมใช้งาน

```
[Main Memory] ---[LOAD]--> [CPU Register]
```

    - **STORE:** ตรงข้ามกับ LOAD คือเมื่อ CPU ทำงานเสร็จแล้วมี "ผลลัพธ์ (Data)" อยู่บนโต๊ะ (Register) คำสั่ง STORE ก็เหมือนการ "นำผลลัพธ์" จากโต๊ะ ไป "เก็บใส่กล่อง" เดิมหรือกล่องใหม่ในโกดัง

```
[CPU Register] ---[STORE]--> [Main Memory]
```

    - **MOVE:** เหมือนการย้ายเอกสารจากลิ้นชักหนึ่งไปยังอีกลิ้นชักหนึ่งบนโต๊ะทำงานเดียวกัน (ย้ายข้อมูลจากรีจิสเตอร์หนึ่งไปยังอีกรีจิสเตอร์หนึ่ง)

```
[Register A] ---[MOVE]--> [Register B]
```

  - **IN/OUT:** เหมือนการรับข้อมูลจาก "เครื่องพิมพ์ (Input Device)" มาวางบนโต๊ะ (IN) หรือส่งข้อมูลจากโต๊ะไป "จอภาพ (Output Device)" (OUT)
- ***ตัวอย่างคำสั่ง:*** *MOV AX, BX (ย้ายค่าจากรีจิสเตอร์ BX ไปยัง AX), LOAD R1, [1000h] (โหลดข้อมูลจากตำแหน่ง 1000h ไปยังรีจิสเตอร์ R1)*

Operation Arithmetic (คำสั่งคำนวณทางคณิตศาสตร์): ทำหน้าที่ดำเนินการทางคณิตศาสตร์กับข้อมูล เช่น

  - **ADD:** บวก
  - **SUBTRACT:** ลบ
  - **MULTIPLY:** คูณ
  - **DIVIDE:** หาร

ตัวอย่างเช่น Operation Arithmetic (คำสั่งคำนวณทางคณิตศาสตร์)

- **แนวคิด:** เปรียบเหมือน "การคำนวณตัวเลข" โดยใช้เครื่องคิดเลข (ALU)
- **การทำงาน:**
  - CPU จะนำข้อมูลตั้งแต่หนึ่งตัวขึ้นไป มาเข้าสู่ "หน่วยคำนวณและตรรกะ (ALU)"
  - ALU จะทำการบวก ลบ คูณ หาร ตามที่ OpCode สั่ง
  - ผลลัพธ์ที่ได้จากการคำนวณจะถูกส่งกลับไปเก็บไว้ในรีจิสเตอร์หรือหน่วยความจำ
- ***ตัวอย่าง:***
  - ***ADD:*** *ADD AX, BX (นำค่าใน BX มาบวกกับ AX แล้วผลลัพธ์เก็บไว้ใน AX)*

```
[Register A] + [Register B] --> [ALU] --> [Register A (Result)]
```

  - ***SUBTRACT:*** *SUB R1, R2 (นำค่าใน R2 มาลบจาก R1 แล้วผลลัพธ์เก็บไว้ใน R1)*
  - ***MULTIPLY, DIVIDE:*** *ทำการคูณหรือหารตามหลักการทางคณิตศาสตร์*
- **Operation Logical (คำสั่งตรรกะ):** ทำหน้าที่ดำเนินการทางตรรกะกับข้อมูล หรือเปรียบเทียบข้อมูล เช่น
  - **AND, OR, NOT, XOR:** ปฏิบัติการบูลีน
  - **COMPARE:** เปรียบเทียบค่าสองค่า

ตัวอย่างเช่น Operation Logical (คำสั่งตรรกะ)

- **แนวคิด:** เปรียบเหมือน "การตัดสินใจ" หรือ "การเปรียบเทียบ" ข้อมูลระดับบิต หรือการใช้กฎทางตรรกะ (AND, OR, NOT)

- **การทำงาน:**
    - CPU จะนำข้อมูลมาเข้าสู่ **ALU** เช่นกัน แต่คราวนี้ ALU จะทำงานในส่วนของตรรกะ
    - **AND, OR, NOT, XOR:** ทำการปฏิบัติการทางบูลีนกับบิตแต่ละบิตของข้อมูล
        - AND (และ): ถ้าบิตทั้งคู่เป็น 1 ผลลัพธ์เป็น 1, นอกนั้นเป็น 0
        - OR (หรือ): ถ้าบิตใดบิตหนึ่งเป็น 1 ผลลัพธ์เป็น 1, นอกนั้นเป็น 0
        - NOT (ไม่): กลับค่าบิตจาก 0 เป็น 1 และ 1 เป็น 0
    - **COMPARE:** เปรียบเทียบค่าสองค่าเพื่อดูว่าเท่ากัน น้อยกว่า หรือมากกว่ากัน ผลลัพธ์มักจะตั้งค่า Flag Register ใน CPU ซึ่งใช้ในการตัดสินใจในคำสั่งถัดไป (เช่น กระโดดถ้าเท่ากัน)

ตัวอย่าง:

- *AND AX, 0FFFh (ใช้ AND เพื่อ Clear บิตบางส่วนใน AX)*

[Register AX (Data)] AND [Mask Value] --> [ALU (Logical Unit)] --> [Register AX (Result)]

- *CMP R1, R2 (เปรียบเทียบค่าใน R1 กับ R2 เพื่อตั้ง Flag)*

9. การทำงานประเภทของ Operation System Control, Operation Input/output และ Operation Conversion

- **Operation System Control (คำสั่งควบคุมระบบ):** ทำหน้าที่ควบคุมการทำงานของระบบโดยรวม หรือสถานะของ CPU เช่น
    - **HALT:** หยุดการทำงานของ CPU
    - **NOOP:** คำสั่งว่างเปล่า ไม่ทำอะไร
    - **ENABLE/DISABLE INTERRUPTS:** เปิด/ปิดการรับสัญญาณขัดจังหวะ
    - **PRIVILEGED INSTRUCTIONS:** คำสั่งพิเศษที่ต้องใช้สิทธิ์เฉพาะ (เช่น คำสั่งเกี่ยวกับระบบปฏิบัติการ)

ตัวอย่างเช่น Operation System Control (คำสั่งควบคุมระบบ)

- **แนวคิด:** เปรียบเหมือน "คำสั่งของผู้บริหาร" ที่สั่งให้ระบบโดยรวมอยู่ในสถานะใดสถานะหนึ่ง หรือจัดการเหตุการณ์สำคัญที่เกิดขึ้น
- **การทำงาน:** คำสั่งเหล่านี้ไม่ได้ทำงานกับข้อมูลโดยตรง แต่มีผลต่อการทำงานของ CPU หรือระบบโดยรวม

- **HALT:** เหมือนการสั่งให้ CPU "หยุดทำงาน" หรือเข้าสู่โหมดประหยัดพลังงาน
- **NOP (No Operation):** "ไม่ทำอะไรเลย" เป็นคำสั่งที่ CPU ประมวลผลแต่ไม่มีผลกระทบใด ๆ ใช้ในการหน่วงเวลา หรือเว้นช่องว่าง
- **SVC (Supervisor Call) / TRAP:** เหมือนการเรียกสายด่วนไปยัง "ผู้ดูแลระบบ (Operating System)" เพื่อขอให้ OS ทำงานบางอย่างที่โปรแกรมผู้ใช้ไม่ได้รับอนุญาตให้ทำโดยตรง (เช่น การเข้าถึงฮาร์ดแวร์โดยตรง, การจัดการหน่วยความจำ)
- **Interrupt-related instructions:** คำสั่งที่ใช้เปิด/ปิดการรับ Interrupt หรือตั้งค่าการตอบสนองต่อ Interrupt

- ***ตัวอย่าง:*** *การทำงานของ CPU เพื่อควบคุมปุ่มหรือสวิตช์ควบคุมสถานะ (On/Off, Sleep Mode) หรือเส้นสัญญาณที่เชื่อมไปยัง "Operating System Kernel" เพื่อขอรับบริการ*

Operation Input/output (คำสั่ง Input/Output): ทำหน้าที่ควบคุมและจัดการการรับส่งข้อมูลระหว่าง CPU กับอุปกรณ์ I/O โดยเฉพาะ เช่น

- **START I/O:** เริ่มต้นการทำงานของอุปกรณ์ I/O
- **TEST I/O:** ตรวจสอบสถานะของอุปกรณ์ I/O
- **READ/WRITE I/O PORT:** อ่าน/เขียนข้อมูลไปยังพอร์ต I/O

ตัวอย่างเช่น Operation Input/output (คำสั่ง Input/Output)

- **แนวคิด:** เปรียบเหมือน "คำสั่งสำหรับการสื่อสาร" ระหว่าง CPU กับโลกภายนอก (อุปกรณ์ต่างๆ)
- **การทำงาน:** คำสั่งเหล่านี้ใช้ในการอ่านข้อมูลจากอุปกรณ์รับเข้า (Input Devices) หรือส่งข้อมูลไปยังอุปกรณ์ส่งออก (Output Devices)
  - **IN:** เหมือนการสั่งให้ CPU "รับ" ข้อมูลจากพอร์ตใดพอร์ตหนึ่งของอุปกรณ์ I/O มาเก็บไว้ในรีจิสเตอร์

```
[Input Device (e.g., Keyboard)] --(Data via I/O Port)--> [CPU Register]
```

  - **OUT:** เหมือนการสั่งให้ CPU "ส่ง" ข้อมูลจากรีจิสเตอร์ไปยังพอร์ตใดพอร์ตหนึ่งของอุปกรณ์ I/O เพื่อแสดงผลหรือควบคุมอุปกรณ์

```
[CPU Register] --(Data via I/O Port)--> [Output Device (e.g., Printer)]
```

  - **Status Check:** บางคำสั่งอาจใช้เพื่อตรวจสอบสถานะของอุปกรณ์ I/O ว่าพร้อมรับ/ส่งข้อมูลหรือไม่

- **ตัวอย่างคำสั่ง:** IN AL, 60h (อ่านข้อมูลจากพอร์ต 60h เข้ามายังรีจิสเตอร์ AL), OUT 20h, AL (ส่งข้อมูลจากรีจิสเตอร์ AL ไปยังพอร์ต 20h)
- ***ตัวอย่าง :*** *เป็นการทำงานของ CPU ที่เชื่อมต่อในการส่งข้อมูลไปยังอุปกรณ์ภายนอกต่างๆ (คีย์บอร์ด, จอภาพ, เครื่องพิมพ์) โดยมีทิศทางการไหลของข้อมูลผ่าน "พอร์ต (Port)"*

- **Operation Conversion (คำสั่งแปลงข้อมูล):** ทำหน้าที่แปลงรูปแบบของข้อมูลจากชนิดหนึ่งไปอีกชนิดหนึ่ง เช่น
    - BINARY_TO_DECIMAL: แปลงเลขฐานสองเป็นฐานสิบ
    - INTEGER_TO_FLOAT: แปลงเลขจำนวนเต็มเป็นทศนิยม
    - ASCII_TO_BINARY: แปลงรหัส ASCII เป็นเลขฐานสอง

  ตัวอย่างเช่น Operation Conversion (คำสั่งแปลงข้อมูล)
    - **แนวคิด:** เปรียบเหมือน "การแปลภาษา" หรือ "การจัดรูปแบบใหม่" ของข้อมูล เพื่อให้เหมาะสมกับการใช้งานหรือการแสดงผล
    - **การทำงาน:** คำสั่งเหล่านี้จะเปลี่ยนรูปแบบการจัดเก็บข้อมูล แต่ค่าของข้อมูลมักจะยังคงเดิม เช่น
        - **Conversion between numerical bases:** แปลงเลขฐาน (เช่น Binary to BCD - Binary Coded Decimal, หรือ BCD to Binary) ซึ่งใช้บ่อยในการแสดงผลตัวเลขบนหน้าจอ
        - **Conversion between data types:** แปลงจาก Integer เป็น Floating-Point หรือกลับกัน (เมื่อต้องการคำนวณที่ต้องการความแม่นยำสูง หรือเมื่อต้องการเก็บเป็นจำนวนเต็ม)
        - **Packed Decimal to Binary / Binary to Packed Decimal:** การแปลงรูปแบบข้อมูลที่ใช้ในการประมวลผลตัวเลขทางการเงิน
    - **ตัวอย่างคำสั่ง:** FILD (โหลด Integer ไปเป็น Floating-Point), FIST (เก็บ Floating-Point เป็น Integer)
    - ***ตัวอย่างการทำงาน:*** *การส่ง "ข้อมูลต้นฉบับ" เข้าไปด้านหนึ่ง ผ่าน "กระบวนการแปลง" เกิดขึ้นภายใน จากนั้นได้ "ข้อมูลรูปแบบใหม่" เช่น "Binary Input" -> "Converter" -> "BCD Output"*

10. การทำงานของ Operation System Control ของคำสั่ง Branch และ คำสั่ง Skip

- **คำสั่ง Branch (หรือ Jump):** เป็นคำสั่งที่เปลี่ยนแปลงลำดับการทำงานปกติของโปรแกรม ทำให้ CPU กระโดดไปปฏิบัติคำสั่งในตำแหน่งอื่นที่ไม่ใช่คำสั่งถัดไปในหน่วยความจำ
    - **Unconditional Branch:** กระโดดไปทันทีโดยไม่มีเงื่อนไข
    - **Conditional Branch:** กระโดดไปเมื่อเงื่อนไขที่กำหนดเป็นจริง (เช่น Branch if Zero, Branch if Not Equal) มักใช้ในการทำ Loop หรือ If-Else

ตัวอย่างเช่น คำสั่ง Branch (หรือ Jump): การกระโดดไปยังที่ใหม่

- **แนวคิด:** เปรียบเหมือนกับการอ่านหนังสืออยู่ แล้วมีข้อความบอกว่า "ถ้า... (เงื่อนไข) ให้ไปอ่านต่อที่หน้า... (เลขหน้าใหม่)" หรือ "ไม่ว่าอะไรจะเกิดขึ้น ให้ไปอ่านต่อที่หน้า... (เลขหน้าใหม่)"
- **การทำงาน:**
    - **CPU จะมีรีจิสเตอร์ Program Counter (PC)** ที่คอยชี้ว่าคำสั่งถัดไปที่จะต้องทำอยู่ที่ใด
    - **ก่อนเจอคำสั่ง Branch:** PC จะชี้ไปที่คำสั่งถัดไปในลำดับปกติ (เช่น PC=100 ชี้ไปคำสั่งที่ 100, พอทำเสร็จ PC=101 ชี้ไปคำสั่งที่ 101)
    - **เมื่อเจอคำสั่ง Branch:**
        - **Unconditional Branch (กระโดดแบบไม่มีเงื่อนไข):** CPU จะเปลี่ยนค่าใน PC ทันที ให้ชี้ไปยังที่อยู่ใหม่ที่ระบุในคำสั่ง Branch นั้นๆ (เช่น JUMP Label_X หมายถึง ไม่ว่าอะไรจะเกิดขึ้น ให้เปลี่ยน PC ไปชี้ที่ Label_X)

```
[Instruction 1]
[Instruction 2]  <-- PC ชี้มาที่นี่
[JUMP Label_X]
[Instruction 3]  <-- คำสั่งนี้ถูกข้าม
[Instruction 4]  <-- คำสั่งนี้ถูกข้าม
...
[Label_X: Instruction 5]  <-- PC ถูกเปลี่ยนมาที่นี่ โปรแกรมทำต่อจากตรงนี้
[Instruction 6]
```

รูปที่ 3-7 ตัวอย่างการเปลี่ยนแปลงลำดับคำสั่งโดยใช้คำสั่งกระโดด (Jump Instruction)

        - **Conditional Branch (กระโดดแบบมีเงื่อนไข):** CPU จะ "ตรวจสอบเงื่อนไข" ที่ตั้งไว้ก่อน (เช่น ผลลัพธ์ของการเปรียบเทียบก่อนหน้าเป็นศูนย์หรือไม่)
            - **ถ้าเงื่อนไขเป็นจริง:** CPU จะเปลี่ยนค่าใน PC ไปยังที่อยู่ใหม่ที่ระบุในคำสั่ง Branch (คล้าย Unconditional Branch)
            - **ถ้าเงื่อนไขเป็นเท็จ:** CPU จะ **ไม่เปลี่ยน** ค่าใน PC แต่จะปล่อยให้ PC เพิ่มขึ้นตามปกติ เพื่อไปทำคำสั่งถัดไปในลำดับเดิม

```
[Instruction 1]
[COMPARE A, B]  <-- CPU เปรียบเทียบ A กับ B ตั้งค่า Flag
[BRANCH_IF_EQUAL Label_Y] <-- PC ชี้มาที่นี่ ตรวจสอบ Flag
[Instruction 2]              <-- ถ้าไม่เท่ากัน PC ไปที่นี่
[Instruction 3]
...
[Label_Y: Instruction 4]  <-- ถ้าเท่ากัน PC จะกระโดดมาที่นี่
[Instruction 5]
```

รูปที่ 3-8 ตัวอย่างคำสั่งกระโดดแบบมีเงื่อนไข (Conditional Branch Instruction Flow)

ความสำคัญ: *ใช้สร้างโครงสร้างการควบคุมโปรแกรมที่ซับซ้อน เช่น if-else (ถ้า...แล้ว...มิฉะนั้น...) และ loop (การทำงานซ้ำๆ)*

<u>คำสั่ง Skip</u>: เป็นคำสั่งที่ข้ามคำสั่งถัดไปในลำดับการทำงานของโปรแกรม หากเงื่อนไขที่กำหนดเป็นจริง

- เป็นคำสั่งแบบมีเงื่อนไข เช่น Skip if Zero ซึ่งถ้าผลลัพธ์เป็นศูนย์ ให้ข้ามคำสั่งถัดไป การข้ามนี้ทำให้เกิดการข้ามคำสั่งหนึ่งคำสั่งโดยไม่ต้องเปลี่ยน Program Counter อย่างชัดเจน

ตัวอย่างเช่น คำสั่ง Skip: การข้ามคำสั่งถัดไป

- **แนวคิด:** เปรียบเหมือนกับการอ่านหนังสืออยู่ แล้วมีข้อความบอกว่า "ถ้า... (เงื่อนไข) ให้ข้ามข้อความถัดไป 1 บรรทัด แล้วไปอ่านต่อบรรทัดที่สองถัดไป"
- **การทำงาน:**
  - คำสั่ง Skip มักจะเป็นแบบมีเงื่อนไข (Conditional Skip)
  - เมื่อ CPU เจอคำสั่ง Skip:
    - **ถ้าเงื่อนไขเป็นจริง:** CPU จะ "เพิ่มค่า PC ไปอีก 2 เท่า" (หรือเพิ่มค่า PC ให้ข้ามคำสั่งถัดไป 1 คำสั่ง) ทำให้คำสั่งที่อยู่ติดกันถูกข้ามไป
    - **ถ้าเงื่อนไขเป็นเท็จ:** CPU จะ **ไม่ทำอะไรกับ PC นอกจากการเพิ่มตามปกติ** ทำให้ PC ชี้ไปยังคำสั่งถัดไปในลำดับเดิม
  - คำสั่ง Skip ไม่ได้ระบุที่อยู่ปลายทางแบบชัดเจนเหมือน Branch แต่เป็นการข้ามไปข้างหน้าแค่ 1 คำสั่งเท่านั้น

```
[Instruction 1]
[CHECK_REGISTER_ZERO]   <-- CPU ตรวจสอบว่ารีจิสเตอร์เป็น 0 หรือไม่
[SKIP_IF_ZERO]          <-- PC ชี้มาที่นี่ ตรวจสอบผลจาก CHECK_REGISTER_ZERO
[Instruction 2]         <-- ถ้าไม่เป็น 0 PC ไปที่นี่, ถ้าเป็น 0 คำสั่งนี้ถูกข้าม
[Instruction 3]         <-- โปรแกรมทำต่อจากตรงนี้ไม่ว่ากรณีใดๆ
```

รูปที่ 3-9 ตัวอย่างคำสั่งข้ามแบบมีเงื่อนไข (Conditional Skip Instruction Flow)

- **ตัวอย่าง:**
- ***ความสำคัญ:*** *ใช้ในการสร้างเงื่อนไขง่ายๆ ที่ต้องการข้ามเพียง 1 คำสั่ง เช่น IF (condition) THEN GOTO next_statement (ถ้าเป็นจริงให้ทำคำสั่งถัดไป, ถ้าไม่จริงให้ข้ามคำสั่งถัดไปแล้วไปทำคำสั่งที่สองถัดไป) มักพบในสถาปัตยกรรม CPU ที่เรียบง่ายกว่า*

11. การทำงาน Operation System Control ของคำสั่ง Procedure Call และ Subroutine Call

- **Procedure Call / Subroutine Call (คำสั่งเรียกใช้โปรซีเดอร์/ซับรูทีน):** เป็นคำสั่งที่ใช้ในการเรียกใช้ชุดคำสั่งย่อย (Subroutine หรือ Procedure) ที่เขียนไว้ต่างหาก เพื่อให้โปรแกรมหลักสามารถนำโค้ดที่ทำงานซ้ำๆ มาใช้ใหม่ได้ โดยไม่จำเป็นต้องเขียนโค้ดซ้ำ
- **การทำงาน:**
    1. เมื่อ CPU พบคำสั่ง Call จะทำการบันทึกที่อยู่ของคำสั่งถัดไปในโปรแกรมหลัก (Return Address) ไว้ในหน่วยความจำชั่วคราว (ส่วนใหญ่จะใช้ Stack)
    2. CPU เปลี่ยนค่าใน Program Counter (PC) ให้ชี้ไปยังที่อยู่เริ่มต้นของ Subroutine นั้น
    3. เมื่อ Subroutine ทำงานเสร็จสิ้น จะมีคำสั่ง **RETURN** ซึ่งจะดึง Return Address ที่บันทึกไว้กลับมาใส่ใน PC ทำให้ CPU กลับไปทำงานต่อที่โปรแกรมหลักในตำแหน่งเดิม

ตัวอย่างเช่น การอธิบายแนวคิดการทำงานของคำสั่ง Procedure Call และ Subroutine Call

คำสั่ง Procedure Call และ Subroutine Call (ซึ่งในทางปฏิบัติมักใช้สลับกันได้ หรือหมายถึงสิ่งเดียวกันในบริบทของภาษาเครื่อง) เป็นคำสั่งประเภท System Control ที่ใช้ในการ **จัดการการเรียกใช้และกลับจากโปรแกรมย่อยหรือฟังก์ชัน**

แนวคิดหลัก: เปรียบเหมือนกับการทำงานหลัก (Main Program) ของเรากำลังทำงานอยู่ แล้วเจอภารกิจย่อย (Subroutine/Procedure) ที่จำเป็นต้องทำ ซึ่งภารกิจย่อยนี้ก็มีขั้นตอนของตัวเอง เมื่อภารกิจย่อยนั้นเสร็จสิ้น เราต้องกลับมาทำงานหลักต่อจากจุดเดิมที่เราค้างไว้

กลไกสำคัญที่เกี่ยวข้อง: Stack (สแตก)

- Stack เป็นพื้นที่หน่วยความจำพิเศษที่ทำงานแบบ **"เข้าทีหลังออกก่อน" (LIFO - Last In, First Out)** เหมือนกับการซ้อนจาน
- CPU จะมีรีจิสเตอร์ตัวหนึ่งชื่อ **Stack Pointer (SP)** ที่คอยชี้ไปยังตำแหน่งบนสุดของ Stack เสมอ
- Stack ใช้ในการบันทึกข้อมูลชั่วคราวที่จำเป็นต้องเก็บไว้ชั่วคราว เช่น ที่อยู่สำหรับกลับ (Return Address)

1. คำสั่ง Procedure Call / Subroutine Call (เช่น CALL)

- **หน้าที่:** ใช้เพื่อ "กระโดด" จากโปรแกรมหลักไปยังจุดเริ่มต้นของโปรแกรมย่อย และ "บันทึก" ตำแหน่งที่จะกลับมา

- **การทำงาน:**
    1. **บันทึก Return Address:** ก่อนที่ CPU จะกระโดดไป CPU จะต้อง **"จำ"** ไว้ก่อนว่า เมื่อโปรแกรมย่อยทำงานเสร็จ จะต้องกลับมาทำงานต่อที่คำสั่งไหนในโปรแกรมหลัก
        - CPU จะนำค่าปัจจุบันของ **Program Counter (PC)** (ซึ่งชี้ไปยังคำสั่งถัดไปหลังจากคำสั่ง CALL) ไป **"Push" (ผลัก)** ลงบน Stack โดยใช้ Stack Pointer
        - นี่คือการบันทึก "ที่อยู่สำหรับกลับ" (Return Address)

```
[Main Program]                          [Stack]
...                                     [Bottom of Stack]
[Instruction A]
[Instruction B] <- PC ชี้ที่นี่ ก่อน CALL       ---(Push PC)---> [Return Address (
[CALL Subroutine_X]
[Instruction C]  <-- นี่คือ Return Address
[Instruction D]
```

รูปที่ 3-10 ตัวอย่างคำสั่งเรียกโปรแกรมย่อยและการจัดการ Stack (Subroutine Call and Stack Operation)

2.กระโดดไปยัง Subroutine: หลังจากบันทึก Return Address แล้ว CPU จะเปลี่ยนค่าใน **Program Counter (PC)** ให้ชี้ไปยัง "ที่อยู่เริ่มต้น" ของ Subroutine ที่ถูกเรียก

```
[Main Program]                          [Subroutine_X]
...                                      [Subroutine_X_Start: Instruction S1]  <
[Instruction B]
[CALL Subroutine_X] ------------->  [Instruction S2]
[Instruction C]                          [Instruction S3]
```

รูปที่ 3-11 ลำดับการทำงานและการส่งต่อคำสั่งไปยังโปรแกรมย่อย (Subroutine Execution Flow)

- ***ตัวอย่างคำสั่ง:*** *CALL MyFunction (เรียกใช้โปรแกรมย่อยชื่อ MyFunction)*

2. คำสั่ง Return (จาก Procedure / Subroutine)

- **หน้าที่:** ใช้เพื่อ "กระโดดกลับ" จากโปรแกรมย่อยไปยังโปรแกรมหลัก หลังจากที่ทำงานของโปรแกรมย่อยเสร็จสิ้น
- **การทำงาน:**
    1. **ดึง Return Address:** เมื่อ CPU มาถึงคำสั่ง RETURN (หรือ RET) ในตอนท้ายของ Subroutine
        - CPU จะทำการ **"Pop" (ดึง)** ค่าที่อยู่บนสุดของ Stack (ซึ่งก็คือ Return Address ที่เคย Push เข้าไป) ออกมา
        - ค่าที่ดึงออกมานี้จะถูกนำไปใส่ใน **Program Counter (PC)**
    2. **กระโดดกลับ:** เมื่อ PC ได้รับค่า Return Address แล้ว CPU ก็จะเริ่มทำงานต่อจากคำสั่งนั้นในโปรแกรมหลัก

```
[Subroutine_X]                      [Stack]
...                                 [Return Address (จาก B)] <- SP ชี้ที่นี่ก่อน RET
[Instruction S3]                    [Bottom of Stack]
[RETURN]          ---(Pop to PC)--->
                                    [Main Program]
                                    ...
                                    [Instruction C]  <-- PC ชี้กลับมาที่นี่
                                    [Instruction D]
```

รูปที่ 3-12 ลำดับการไหลของโปรแกรมในลักษณะการเรียกใช้โปรแกรมย่อยซ้ำ (Repeated Subroutine Execution Flow)

ตัวอย่างคำสั่ง: *RET*

สรุป Flow การไหลของโปรแกรมแบบมี Call/Return:

1. *โปรแกรมหลักทำงานตามลำดับ (Instr A -> Instr B)*
2. *เจอคำสั่ง CALL Subroutine_X ที่ Instr B*
3. ***PC ของ Instr C (Return Address) ถูก Push ลง Stack***
4. ***PC ถูกเปลี่ยนไปชี้ที่ Subroutine_X_Start***
5. *โปรแกรมย่อย Subroutine_X ทำงาน (Instr S1 -> Instr S2 -> Instr S3)*
6. *เจอคำสั่ง RETURN ใน Subroutine_X*
7. ***Return Address ถูก Pop ออกจาก Stack ไปใส่ PC***
8. ***PC กลับไปชี้ที่ Instr C***
9. *โปรแกรมหลักทำงานต่อจาก Instr C -> Instr D*

12. การทำงาน Operation System Control ของคำสั่งควบคุมอุปกรณ์ Input/Output

แนวคิด: คำสั่งควบคุมอุปกรณ์ I/O เปรียบเสมือน "คำสั่งสั่งการ" ที่ CPU ส่งไปยัง "คนกลาง" (I/O Module หรือ I/O Controller) ที่ทำหน้าที่ดูแลอุปกรณ์ I/O ต่างๆ เช่น คีย์บอร์ด, เมาส์, เครื่องพิมพ์, ฮาร์ดดิสก์, จอภาพ เพื่อบอกให้คนกลางเหล่านี้ทำงานเฉพาะเจาะจง หรือรับรู้สถานะของอุปกรณ์

ทำไมถึงต้องมีคำสั่งควบคุม I/O แยกต่างหาก?

- **ความเร็วที่แตกต่างกัน:** CPU ทำงานเร็วมาก แต่อุปกรณ์ I/O ทำงานช้ากว่ามาก การสื่อสารโดยตรงจะทำให้ CPU ต้องรอเสียเวลา
- **ความหลากหลายของอุปกรณ์:** อุปกรณ์ I/O มีหลายประเภท แต่ละประเภทมีวิธีการทำงานที่แตกต่างกัน
- **ความซับซ้อน:** การจัดการ I/O มีความซับซ้อน เช่น การจัดการบัฟเฟอร์ข้อมูล, การจัดการข้อผิดพลาด, การตอบสนองต่อ Interrupt

ดังนั้น CPU จึงไม่ได้สื่อสารกับอุปกรณ์ I/O โดยตรง แต่จะสื่อสารผ่าน **I/O Module (โมดูล I/O)** หรือ **I/O Controller (ตัวควบคุม I/O)** ซึ่งเป็นวงจรอิเล็กทรอนิกส์พิเศษที่เชื่อมต่อระหว่าง System Bus ของคอมพิวเตอร์กับอุปกรณ์ I/O นั้นๆ

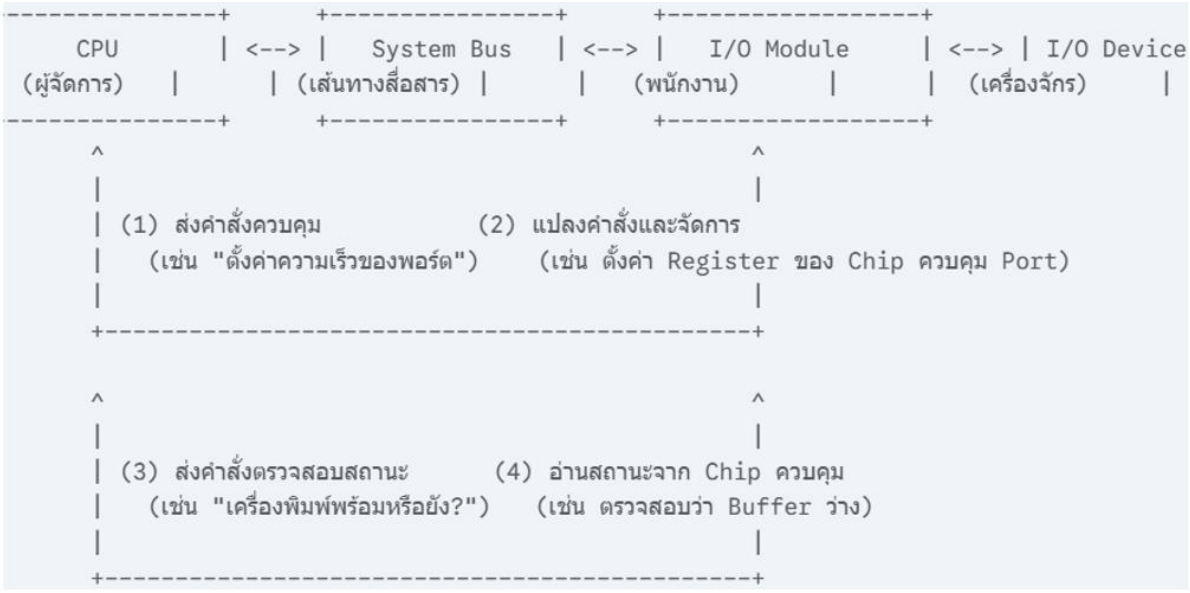
ประเภทของคำสั่งควบคุมอุปกรณ์ I/O ที่ CPU ส่งออกไป:

คำสั่งเหล่านี้มักไม่ได้ทำหน้าที่ "ย้ายข้อมูล" โดยตรง (ซึ่งจะเป็นหน้าที่ของคำสั่ง IN หรือ OUT) แต่เป็นคำสั่งที่บอกให้ I/O Module/Controller ทำบางสิ่ง หรือสอบถามสถานะ:

1. **คำสั่งควบคุม (Control Commands):**
    - **แนวคิด:** เหมือนการสั่งให้คนกลาง "ตั้งค่า" หรือ "เตรียมพร้อม" อุปกรณ์
    - **การทำงาน:** CPU จะส่งคำสั่งควบคุมไปยัง I/O Module เพื่อสั่งให้อุปกรณ์ I/O ทำงานเฉพาะอย่าง เช่น:
        - **Reset:** สั่งให้รีเซ็ตอุปกรณ์ (เช่น รีเซ็ตเครื่องพิมพ์)
        - **Start/Stop:** สั่งให้อุปกรณ์เริ่มทำงาน (เช่น เริ่มหมุนจานของฮาร์ดดิสก์) หรือหยุดทำงาน
        - **Mode Setting:** ตั้งค่าโหมดการทำงานของอุปกรณ์ (เช่น กำหนดความละเอียดของหน้าจอ, ตั้งค่าพารามิเตอร์ของพอร์ตสื่อสาร)
    - ***ตัวอย่าง:*** *CPU อาจส่งค่าบางอย่างไปยัง "Control Register" ของ I/O Module เพื่อกำหนดความเร็วของพอร์ตอนุกรม (Serial Port)*
2. **คำสั่งตรวจสอบสถานะ (Test/Status Commands):**
    - **แนวคิด:** เหมือนการสอบถามคนกลางว่า "อุปกรณ์นั้นตอนนี้เป็นอย่างไรบ้าง?" หรือ "พร้อมทำงานหรือยัง?"
    - **การทำงาน:** CPU จะส่งคำสั่งเพื่ออ่านค่าจาก "Status Register" ของ I/O Module
        - **Ready/Busy:** ตรวจสอบว่าอุปกรณ์พร้อมที่จะรับข้อมูล (Ready) หรือยังคงทำงานอยู่ (Busy)
        - **Error:** ตรวจสอบว่ามีข้อผิดพลาดเกิดขึ้นกับอุปกรณ์หรือไม่ (เช่น กระดาษหมดในเครื่องพิมพ์)
        - **Data Available:** ตรวจสอบว่ามีข้อมูลจากอุปกรณ์พร้อมให้อ่านแล้วหรือยัง (เช่น มีการกดแป้นพิมพ์หรือไม่)
    - ***ความสำคัญ:*** *ช่วยให้ CPU ไม่ต้องรออุปกรณ์โดยไม่จำเป็น และสามารถจัดการกับข้อผิดพลาดได้*

ตัวอย่างการทำงานร่วมกัน :

สมุติว่า CPU เป็น "ผู้จัดการ" ที่ต้องการสั่งงาน "พนักงาน" (I/O Module) ให้จัดการกับ "เครื่องจักร" (I/O Device)



รูปที่ 3-13 แผนผังโครงสร้างการควบคุมและการสื่อสารระหว่าง CPU และโมดูลอินพุต/เอาต์พุต

คำอธิบายเพิ่มเติมสำหรับการไหลของข้อมูล:

1. ***CPU** ต้องการควบคุมหรือสอบถามอุปกรณ์ I/O*
2. ***CPU** จะนำคำสั่งควบคุม หรือที่อยู่ของ Register สถานะ/ควบคุมของ I/O Module ไปวางบน **Address Bus***
3. ***CPU** จะส่งข้อมูลที่จะใช้ในการควบคุม (ถ้ามี) ไปบน **Data Bus** และส่งสัญญาณควบคุม (เช่น Write signal) บน **Control Bus***
4. ***I/O Module** ที่ถูกระบุที่อยู่ จะรับคำสั่งและข้อมูลนั้น*
5. ***I/O Module** จะทำการแปลงคำสั่งของ CPU ให้เป็นสัญญาณไฟฟ้าที่เหมาะสมกับอุปกรณ์ I/O นั้นๆ และส่งไปยังอุปกรณ์*
6. ***I/O Module** ยังสามารถอ่านสถานะจากอุปกรณ์ I/O แล้วนำมาเก็บไว้ใน Status Register ของตนเอง เพื่อให้ CPU สามารถอ่านข้อมูลสถานะนี้กลับไปได้*

***ความสำคัญ:** คำสั่งเหล่านี้ทำให้ CPU สามารถ "ดูแล" และ "ประสานงาน" กับอุปกรณ์ I/O ได้อย่างเป็นระบบ ช่วยให้โปรแกรมสามารถโต้ตอบกับผู้ใช้ (ผ่านคีย์บอร์ด/จอภาพ) จัดเก็บและเรียกข้อมูล (ผ่านฮาร์ดดิสก์) และควบคุมอุปกรณ์ภายนอกอื่นๆ ได้อย่างมีประสิทธิภาพ*

13. การเขียนรหัสคำสั่งแบบ 4-Address Instruction, 3-Address Instruction, 2-Address Instruction และแบบ 1-Address Instruction ซึ่งมีรายละเอียดดังนี้:

- **4-Address Instruction:** คำสั่งที่มี 4 ส่วนสำหรับระบุที่อยู่ของ Operand สองตัว, ที่อยู่สำหรับเก็บผลลัพธ์ และที่อยู่ของคำสั่งถัดไป (Opcode Dest, Src1, Src2, Next_Instr_Addr)
- **3-Address Instruction:** คำสั่งที่มี 3 ส่วนสำหรับระบุที่อยู่ของ Operand สองตัว และที่อยู่สำหรับเก็บผลลัพธ์ (หรือ Destination) โดยคำสั่งถัดไปจะถูกกำหนดโดย Program Counter (Opcode Dest, Src1, Src2)
- **2-Address Instruction:** คำสั่งที่มี 2 ส่วนสำหรับระบุที่อยู่ โดยที่ส่วนหนึ่งทำหน้าที่เป็นทั้ง Source และ Destination อีกส่วนหนึ่งเป็น Source (Opcode Dest/Src1, Src2)
- **1-Address Instruction:** คำสั่งที่มี 1 ส่วนสำหรับระบุที่อยู่ของ Operand อีกตัวหนึ่งจะถูกอ้างอิงจากรีจิสเตอร์สะสม (Accumulator) โดยปริยาย (Opcode Src/Dest)

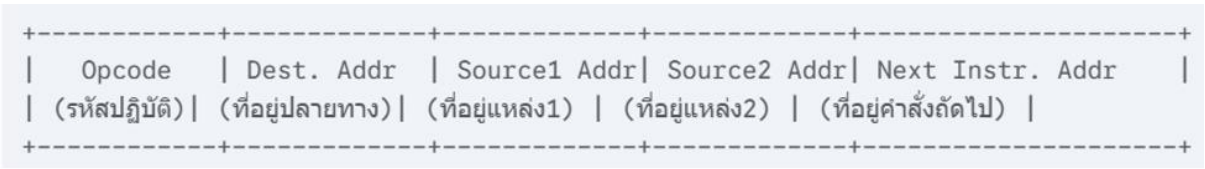
การอธิบายแนวคิดการเขียนรหัสคำสั่งแบบต่างๆ

การออกแบบรูปแบบคำสั่ง (Instruction Format) เป็นการกำหนดว่าคำสั่งหนึ่งคำสั่งจะถูกแบ่งออกเป็นกี่ช่อง (Field) และแต่ละช่องใช้เก็บข้อมูลอะไรบ้าง โดยเฉพาะส่วนที่ระบุ "ที่อยู่" (Address) ของข้อมูลที่คำสั่งนั้นจะทำงานด้วย จำนวน Address ที่สามารถระบุได้ในคำสั่งเดียว ทำให้เกิดการแบ่งประเภทของ Instruction Format

ลองนึกภาพแต่ละคำสั่งเป็น "การ์ดคำสั่ง" ที่มีช่องให้เราเติมข้อมูลลงไปครับ

1. 4-Address Instruction (คำสั่งแบบ 4 ที่อยู่)

- **แนวคิด:** เป็นคำสั่งที่ "บอกทุกอย่างในคำสั่งเดียว" คือ บอกแหล่งข้อมูล 2 ตัว, บอกที่เก็บผลลัพธ์ และบอกคำสั่งถัดไป
- **โครงสร้าง (จินตนาการถึงการ์ด):**



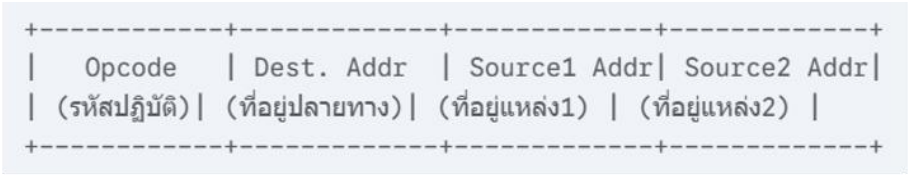
รูปที่ 3-14 โครงสร้างการควบคุมและการสื่อสารระหว่าง CPU และโมดูลอินพุต/เอาต์พุต แบบ 4 ที่อยู่

- **การทำงาน:** CPU ทำตาม Opcode โดยใช้ข้อมูลจาก Source1 และ Source2 เก็บผลลัพธ์ที่ Dest. และหลังจากนั้น PC (Program Counter) จะถูกเปลี่ยนไปชี้ที่ Next Instr. Addr ที่ระบุในคำสั่งเลย ไม่ต้องเพิ่มอัตโนมัติ
- **ข้อดี:** โปรแกรมมีความชัดเจนว่าไปไหนต่อ ไม่ต้องพึ่ง PC
- **ข้อเสีย:** คำสั่งมีขนาดใหญ่มาก เปลืองหน่วยความจำ และซับซ้อนในการประมวลผล ไม่นิยมใช้ใน CPU สมัยใหม่ (มีแต่ในเชิงแนวคิดหรือในยุคแรกๆ มากๆ)

- **ตัวอย่างเชิงแนวคิด:** ADD R1, R2, R3, LABEL_NEXT (นำค่า R2 + R3 ไปเก็บ R1 แล้วไปทำคำสั่งที่ LABEL_NEXT ต่อ)

2. 3-Address Instruction (คำสั่งแบบ 3 ที่อยู่)

- **แนวคิด:** เป็นคำสั่งที่ "บอกแหล่งข้อมูล 2 ตัว และที่เก็บผลลัพธ์" โดยปกติคำสั่งถัดไปจะอยู่ติดกัน
- **โครงสร้าง (จินตนาการถึงการ์ด):**



รูปที่ 3-15 โครงสร้างการควบคุมและการสื่อสารระหว่าง CPU และโมดูลอินพุต/เอาต์พุต แบบ 3 ที่อยู่

- **การทำงาน:** CPU ทำตาม Opcode โดยใช้ข้อมูลจาก Source1 และ Source2 เก็บผลลัพธ์ที่ Dest. หลังจากนั้น PC จะเพิ่มขึ้นอัตโนมัติเพื่อชี้ไปยังคำสั่งถัดไปในลำดับปกติ (เว้นแต่จะเป็นคำสั่ง Jump/Branch)
- **ข้อดี:** ยืดหยุ่นในการใช้งาน สามารถทำงานกับข้อมูล 2 ตัวและเก็บผลลัพธ์ในที่ที่ 3 ได้ โปรแกรมสั้นและอ่านง่าย
- **ข้อเสีย:** คำสั่งยังคงมีขนาดค่อนข้างใหญ่
- **ตัวอย่าง:** ADD R1, R2, R3 (นำค่า R2 + R3 ไปเก็บใน R1), MOV R1, M[1000] (ย้ายข้อมูลจากหน่วยความจำตำแหน่ง 1000 ไป R1)
    - (R1, R2, R3 อาจหมายถึงรีจิสเตอร์ หรือที่อยู่หน่วยความจำ)

3. 2-Address Instruction (คำสั่งแบบ 2 ที่อยู่)

- **แนวคิด:** เป็นคำสั่งที่ "บอกแหล่งข้อมูล 2 ตัว โดยที่หนึ่งในนั้นจะเป็นที่เก็บผลลัพธ์ด้วย"
- **โครงสร้าง (จินตนาการถึงการ์ด):**

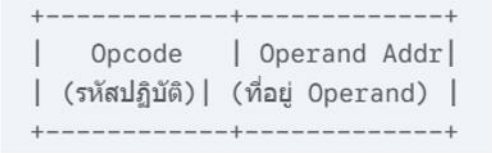


รูปที่ 3-16 โครงสร้างการควบคุมและการสื่อสารระหว่าง CPU และโมดูลอินพุต/เอาต์พุต แบบ 2 ที่อยู่

- **การทำงาน:** CPU ทำตาม Opcode โดยใช้ข้อมูลจาก Dest./Source1 และ Source2 ผลลัพธ์จะถูกเก็บกลับไปที่ Dest./Source1 นั้นๆ (คือหนึ่งใน Operand จะถูกเขียนทับด้วยผลลัพธ์) PC จะเพิ่มขึ้นอัตโนมัติ
- **ข้อดี:** คำสั่งมีขนาดเล็กลง ประหยัดพื้นที่
- **ข้อเสีย:** Operand ตัวหนึ่งจะถูกเขียนทับ ทำให้ต้องมีการย้ายข้อมูลก่อนหากต้องการเก็บค่าเดิมไว้ และบางครั้งอาจต้องใช้คำสั่งเพิ่มขึ้นเพื่อย้ายข้อมูลไปมาระหว่าง Register หรือ Memory
- **ตัวอย่าง:** ADD R1, R2 (นำค่า R2 มาบวกกับ R1 แล้วผลลัพธ์เก็บใน R1), MOV R1, R2 (ย้ายค่าจาก R2 ไป R1)

4. 1-Address Instruction (คำสั่งแบบ 1 ที่อยู่)

- **แนวคิด:** เป็นคำสั่งที่ "บอกแหล่งข้อมูลเพียง 1 ตัว" ส่วนอีกตัวหนึ่งและที่เก็บผลลัพธ์จะถูกอ้างอิงจากรีจิสเตอร์พิเศษที่เรียกว่า **Accumulator (AC)** โดยอัตโนมัติ
- **โครงสร้าง (จินตนาการถึงการ์ด):**



รูปที่ 3-17 โครงสร้างการควบคุมและการสื่อสารระหว่าง CPU และโมดูลอินพุต/เอาต์พุต แบบ 1 ที่อยู่

- **การทำงาน:**
  - ถ้าเป็นคำสั่ง ADD: CPU จะนำค่าจาก Operand Addr มาบวกกับค่าใน Accumulator แล้วผลลัพธ์เก็บใน Accumulator
    - ADD X หมายถึง AC = AC + M[X] (ค่าใน Accumulator = ค่าใน Accumulator + ค่าในหน่วยความจำที่ตำแหน่ง X)
  - ถ้าเป็นคำสั่ง LOAD: CPU จะนำค่าจาก Operand Addr มาใส่ใน Accumulator
    - LOAD X หมายถึง AC = M[X]
  - ถ้าเป็นคำสั่ง STORE: CPU จะนำค่าจาก Accumulator ไปเก็บใน Operand Addr
    - STORE X หมายถึง M[X] = AC
- **ข้อดี:** คำสั่งมีขนาดเล็กที่สุด ประหยัดพื้นที่หน่วยความจำ
- **ข้อเสีย:** การคำนวณที่ซับซ้อนจะต้องใช้คำสั่งจำนวนมาก และมีการย้ายข้อมูลเข้าออก Accumulator บ่อยครั้ง ทำให้โปรแกรมยาวขึ้นและอาจช้าลง (เพราะต้องมีการโหลด/เก็บข้อมูลบ่อยกว่า)
- **ตัวอย่าง:** ADD B (นำค่าในตำแหน่ง B มาบวกกับค่าใน Accumulator)

สรุปความสัมพันธ์ของ Address กับประสิทธิภาพ/ความซับซ้อน:

- ***จำนวน Address มาก:*** *คำสั่งมีความสมบูรณ์ในตัว ประมวลผลได้ในคำสั่งเดียวน้อยลง ทำให้โปรแกรมโดยรวมสั้นลง แต่ตัวคำสั่งมีขนาดใหญ่และอาจต้องใช้ Control Unit ที่ซับซ้อนขึ้น*
- ***จำนวน Address น้อย:*** *คำสั่งมีขนาดเล็ก ประหยัดพื้นที่ แต่การทำงานที่ซับซ้อนต้องใช้คำสั่งหลายบรรทัด ทำให้โปรแกรมยาวขึ้น และ CPU ต้องเข้าถึงหน่วยความจำบ่อยขึ้นเพื่อโหลดข้อมูล*

การเลือกใช้รูปแบบใดขึ้นอยู่กับสถาปัตยกรรมของ CPU และเป้าหมายในการออกแบบครับ ปัจจุบัน CPU ส่วนใหญ่จะใช้รูปแบบ 3-Address หรือ 2-Address และมีการใช้รีจิสเตอร์ภายในจำนวนมากเพื่อลดการอ้างอิงหน่วยความจำโดยตรง

14. การทำงานของคำสั่ง Immediate Addressing Mode และ Register Addressing Mode

- **Immediate Addressing Mode (การกำหนดแอดเดรสแบบทันที):**
  - **การทำงาน:** ข้อมูล Operand ที่จะใช้ในการปฏิบัติการนั้นถูกบรรจุอยู่ "โดยตรง" ภายในตัวคำสั่งนั้นเอง
  - **ตัวอย่าง:** ADD A, 5 (บวกค่า 5 เข้าไปในรีจิสเตอร์ A) ในที่นี้ 5 คือข้อมูลที่อยู่ภายในคำสั่งเลย ไม่ได้อ้างอิงจากที่อื่น
  - **ข้อดี:** เข้าถึงข้อมูลได้เร็วที่สุด เพราะไม่ต้องเข้าถึงหน่วยความจำเพิ่มเติม
- **Register Addressing Mode (การกำหนดแอดเดรสแบบรีจิสเตอร์):**
  - **การทำงาน:** Operand ที่จะใช้ในการปฏิบัติการนั้นถูกเก็บอยู่ในรีจิสเตอร์ภายใน CPU
  - **ตัวอย่าง:** ADD R1, R2 (บวกค่าในรีจิสเตอร์ R2 เข้าไปในรีจิสเตอร์ R1) ในที่นี้ R1 และ R2 เป็นชื่อรีจิสเตอร์
  - **ข้อดี:** เข้าถึงข้อมูลได้รวดเร็วมากเช่นกัน เนื่องจากรีจิสเตอร์อยู่ใกล้ CPU และเข้าถึงโดยตรง

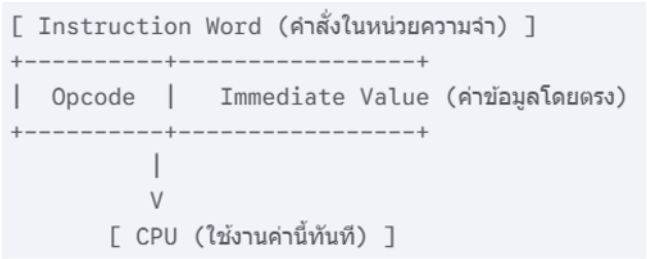
การอธิบายแนวคิดการทำงานของ Immediate Addressing Mode และ Register Addressing Mode

Addressing Mode คือวิธีการที่ CPU ใช้ในการระบุ "ที่อยู่" หรือ "ตำแหน่ง" ของข้อมูล (Operand) ที่คำสั่งนั้นๆ ต้องการนำไปใช้งาน การเลือกใช้ Addressing Mode ที่เหมาะสมจะส่งผลต่อประสิทธิภาพและความยืดหยุ่นของโปรแกรม

1. Immediate Addressing Mode (การกำหนดแอดเดรสแบบทันที)

- **แนวคิด:** เปรียบเหมือนกับการมี "ตัวเลข" อยู่ใน "กระดาษโน้ตของคำสั่ง" ทันที ไม่ต้องไปหาที่ไหนเพิ่ม
- **คำอธิบาย:** ในโหมดนี้ **ตัว Operand เองคือค่าข้อมูลที่แท้จริง** และถูกบรรจุ (embedded) อยู่ในส่วนของ Operand Field ของคำสั่งเลยโดยตรง ไม่ใช่ที่อยู่ของข้อมูล และไม่ได้ชี้ไปยังที่อื่น
- **การทำงาน:**
  - เมื่อ CPU ดึงคำสั่งที่มี Immediate Operand เข้ามา
  - CPU จะตีความ OpCode และพบว่า Operand Field นั้นไม่ได้เป็น Address แต่เป็น *ค่าข้อมูล* ที่ต้องการใช้ทันที

- CPU สามารถนำค่านั้นไปใช้ในการคำนวณหรือปฏิบัติการได้โดยตรง โดยไม่ต้องเข้าถึงหน่วยความจำหลักหรือรีจิสเตอร์อื่นๆ เพื่อ "หยิบ" ข้อมูล



รูปที่ 3-18 การระบุตำแหน่งข้อมูลแบบทันที (Immediate Addressing Mode)

- **ข้อดี:**
  - **เร็วที่สุด:** เพราะไม่ต้องเสียเวลาในการคำนวณที่อยู่หรือเข้าถึงหน่วยความจำ/รีจิสเตอร์อื่นเพื่อดึงข้อมูล ข้อมูลพร้อมใช้ทันที
  - **ง่ายต่อการเขียนโปรแกรม:** สำหรับค่าคงที่หรือตัวเลขที่ไม่เปลี่ยนแปลง
- **ข้อเสีย:**
  - **ค่าที่ใช้มีขนาดจำกัด:** ขนาดของค่า Immediate Operand จะถูกจำกัดด้วยขนาดของ Operand Field ใน Instruction Format (เช่น ถ้า Operand Field มี 8 บิต ก็เก็บค่าได้แค่ 0-255)
  - **ไม่ยืดหยุ่น:** ใช้ได้เฉพาะกับค่าคงที่ ไม่สามารถใช้กับข้อมูลที่เปลี่ยนแปลงได้
- *ตัวอย่างคำสั่ง (Assembler Syntax):*
  - *ADD R1, #5 (หรือ ADD R1, 5) : นำค่า 5 มาบวกกับค่าในรีจิสเตอร์ R1 แล้วเก็บผลลัพธ์ใน R1*
  - *MOV R2, #100 : นำค่า 100 มาเก็บในรีจิสเตอร์ R2*

2. Register Addressing Mode (การกำหนดแอดเดรสแบบรีจิสเตอร์)

- **แนวคิด:** เปรียบเหมือนกับการที่ "คำสั่ง" ไม่ได้มีตัวเลขอยู่ด้วย แต่ "คำสั่ง" บอกว่า "ให้ไปดูที่ลิ้นชัก A บนโต๊ะทำงานของฉัน (Register A) นะ ข้อมูลที่ต้องการอยู่ในนั้น"
- **คำอธิบาย:** ในโหมดนี้ **Operand Field ของคำสั่งจะระบุ "ชื่อ" หรือ "หมายเลข" ของรีจิสเตอร์** ภายใน CPU ที่เก็บข้อมูลที่ต้องการใช้งานอยู่
- **การทำงาน:**
  - เมื่อ CPU ดึงคำสั่งที่มี Register Operand เข้ามา
  - CPU จะตีความ OpCode และพบว่า Operand Field ชี้ไปยังหมายเลขรีจิสเตอร์
  - CPU สามารถเข้าถึงข้อมูลในรีจิสเตอร์ที่ระบุได้ทันที (เพราะรีจิสเตอร์เป็นหน่วยความจำที่เร็วที่สุดใน CPU) และนำค่านั้นไปใช้ในการคำนวณหรือปฏิบัติการ

- **ข้อดี:**
  - **เร็วมาก:** รองจาก Immediate Mode เพราะข้อมูลอยู่ใน CPU อยู่แล้ว ไม่ต้องเข้าถึงหน่วยความจำหลัก
  - **ยืดหยุ่น:** สามารถใช้กับข้อมูลที่เปลี่ยนแปลงได้ (เพราะข้อมูลในรีจิสเตอร์สามารถเปลี่ยนได้)
  - **คำสั่งสั้น:** เพราะการระบุชื่อรีจิสเตอร์ใช้บิตน้อยกว่าการระบุที่อยู่หน่วยความจำหลัก
- **ข้อเสีย:**
  - **จำนวนรีจิสเตอร์จำกัด:** มีรีจิสเตอร์ให้ใช้จำกัด หากข้อมูลที่ต้องการมีจำนวนมากเกินรีจิสเตอร์ ต้องมีการย้ายข้อมูลไปมาระหว่างรีจิสเตอร์กับหน่วยความจำ
- *ตัวอย่างคำสั่ง (Assembler Syntax):*
  - *ADD R1, R2 : นำค่าในรีจิสเตอร์ R2 มาบวกกับค่าในรีจิสเตอร์ R1 แล้วเก็บผลลัพธ์ใน R1*
  - *MOV R3, R4 : ย้ายค่าจากรีจิสเตอร์ R4 ไปเก็บในรีจิสเตอร์ R3*

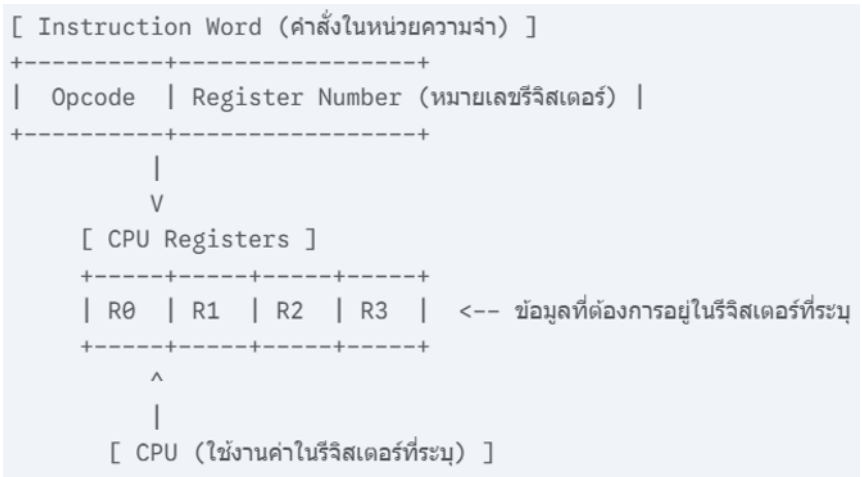
สรุปความแตกต่างที่สำคัญ:

- *Immediate: ข้อมูล **อยู่ในตัวคำสั่ง** เลย (ค่าคงที่)*
- *Register: ข้อมูล **อยู่ในรีจิสเตอร์** ที่คำสั่งชี้ไป (ตัวแปร)*

ทั้งสองโหมดนี้เป็นโหมดที่เร็วที่สุดในการเข้าถึง Operand เนื่องจากไม่มีการเข้าถึงหน่วยความจำหลัก

15. การทำงานของคำสั่ง Direct Addressing Mode และ Register Indirect Addressing Mode

- Direct Addressing Mode (การกำหนดแอดเดรสแบบโดยตรง):
  - **การทำงาน:** ส่วนของ Operand ในคำสั่งจะระบุ "ที่อยู่จริง" ของข้อมูลในหน่วยความจำ1
  - **ตัวอย่าง:** LOAD A, 1000 (โหลดข้อมูลที่เก็บอยู่ในตำแหน่งหน่วยความจำหมายเลข 1000 มาไว้ในรีจิสเตอร์ A) CPU สามารถเข้าถึงข้อมูลได้โดยตรงจากที่อยู่



รูปที่ 3-19 การระบุตำแหน่งข้อมูลโดยตรง (Direct Addressing Mode)

  - **ข้อดี:** เข้าใจง่าย

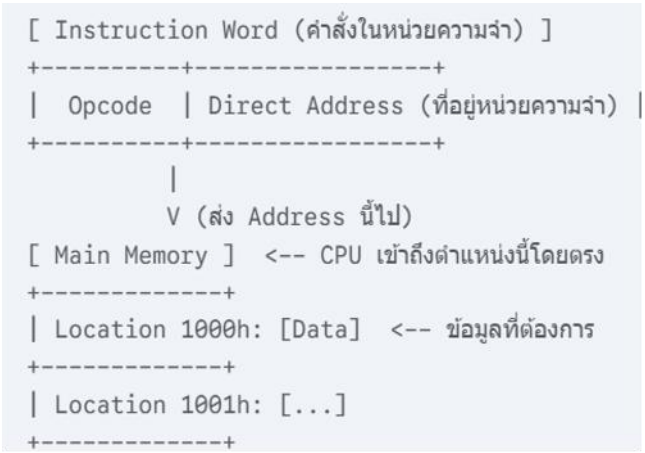
- **Register Indirect Addressing Mode (การกำหนดแอดเดรสแบบรีจิสเตอร์ทางอ้อม):**
    - **การทำงาน:** ส่วนของ Operand ในคำสั่งจะระบุ "ชื่อรีจิสเตอร์" ซึ่งภายในรีจิสเตอร์นั้นจะบรรจุ "ที่อยู่ของข้อมูล" ที่แท้จริงในหน่วยความจำ
    - **ตัวอย่าง:** LOAD A, (R1) (โหลดข้อมูลจากที่อยู่ซึ่งเก็บอยู่ในรีจิสเตอร์ R1 มาไว้ในรีจิสเตอร์ A) CPU ต้องอ่านค่าจาก R1 ก่อน เพื่อนำไปหาที่อยู่ในหน่วยความจำอีกครั้งหนึ่ง
    - **ข้อดี:** มีความยืดหยุ่นกว่า Direct Addressing ในการเข้าถึงข้อมูลที่ต่อเนื่องกัน

สรุปการอธิบายแนวคิดการทำงานของ Direct Addressing Mode และ Register Indirect Addressing Mode

Addressing Mode เป็นวิธีการที่ CPU ใช้ในการค้นหา "ตำแหน่ง" ของข้อมูล (Operand) ที่คำสั่งนั้นๆ ต้องการนำไปใช้งาน โหมดเหล่านี้เกี่ยวข้องกับการเข้าถึงหน่วยความจำหลัก ซึ่งเป็นส่วนสำคัญของระบบคอมพิวเตอร์

1. Direct Addressing Mode (การกำหนดแอดเดรสแบบโดยตรง)

- **แนวคิด:** เปรียบเหมือนกับการมี "บัตรเชิญ" ที่ระบุ "เลขที่บ้าน" ของข้อมูล (Operand) ไว้บนบัตรนั้นโดยตรง ไม่ต้องผ่านใคร หรือไปหาจากที่อื่นอีก
- **คำอธิบาย:** ในโหมดนี้ **Operand Field ของคำสั่งจะบรรจุ "ที่อยู่จริง" (Effective Address) ของข้อมูลในหน่วยความจำหลักโดยตรง** เมื่อ CPU ประมวลผลคำสั่งนี้ ก็จะนำค่า Address ที่ได้จากคำสั่ง ไปใช้เข้าถึงข้อมูลในหน่วยความจำได้ทันที
- **การทำงาน:**
    1. CPU ดึงคำสั่งที่มี Direct Address เข้ามา
    2. CPU ตีความ OpCode และพบว่า Operand Field นั้นคือ **ที่อยู่ของหน่วยความจำ**
    3. CPU นำที่อยู่นั้นไปวางบน **Address Bus**



รูปที่ 3-20 กลไกการเข้าถึงข้อมูลในหน่วยความจำหลักแบบระบุตำแหน่งโดยตรง (Direct Addressing Mechanism)

4. **Memory Unit** อ่านที่อยู่นั้น และส่งข้อมูลจากตำแหน่งนั้นกลับมายัง CPU ผ่าน **Data Bus**

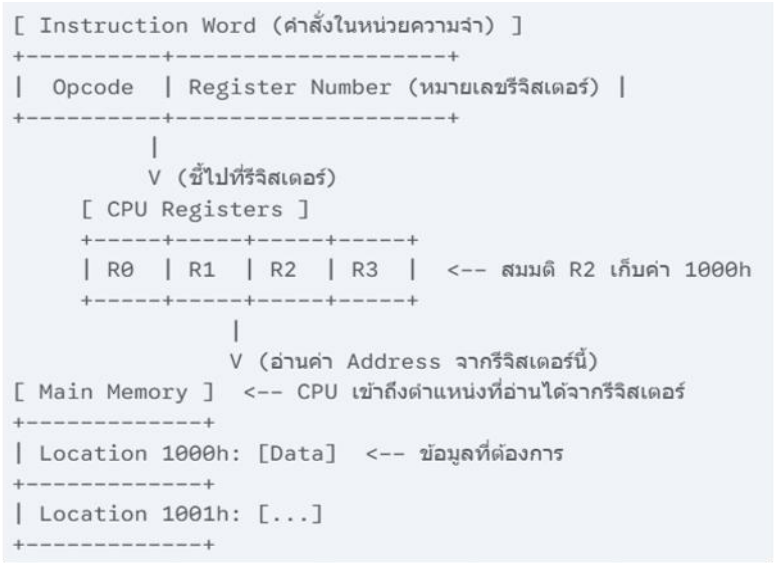
ข้อดี:

- **ง่ายต่อการเข้าใจและใช้งาน:** เพราะคำสั่งบอกที่อยู่ของข้อมูลโดยตรง
- **เร็วพอสมควร:** เพราะไม่ต้องมีการคำนวณที่อยู่เพิ่มเติม (ต่างจากโหมดที่ซับซ้อนกว่านี้)

- **ข้อเสีย:**
    - **ขนาดของคำสั่งยาวขึ้น:** หากหน่วยความจำมีขนาดใหญ่ (เช่น 4GB, 8GB) ที่อยู่ก็จะใช้บิตจำนวนมาก ทำให้ Operand Field ในคำสั่งต้องมีขนาดใหญ่ตามไปด้วย
    - **ไม่ยืดหยุ่นในการเขียนโปรแกรม:** ไม่สามารถเปลี่ยนที่อยู่ของข้อมูลได้ง่ายๆ ในขณะรันโปรแกรม (เช่น เข้าถึงข้อมูลใน Array ที่ตำแหน่งต่างๆ กัน)
- *ตัวอย่างคำสั่ง (Assembler Syntax):*
    - *LOAD R1, [1000h] โหลดข้อมูลจากหน่วยความจำ**ตำแหน่ง 1000h** มาเก็บในรีจิสเตอร์ R1*
    - *STORE [2000h], R2 : เก็บข้อมูลจากรีจิสเตอร์ R2 ไปยังหน่วยความจำที่ **ตำแหน่ง 2000h***

2. Register Indirect Addressing Mode (การกำหนดแอดเดรสแบบรีจิสเตอร์ทางอ้อม)

- **แนวคิด:** เปรียบเหมือนกับการมี "บัตรเชิญ" ที่ไม่ได้บอกเลขที่บ้านโดยตรง แต่บอกว่า "ให้ไปดูที่ลิ้นชัก A บนโต๊ะทำงานของฉัน (Register A) นะ ในนั้นมีกระดาษอีกแผ่นที่เขียนเลขที่บ้านของข้อมูลที่ต้องการไว้"
- **คำอธิบาย:** ในโหมดนี้ **Operand Field ของคำสั่งจะระบุ "ชื่อ" หรือ "หมายเลข" ของรีจิสเตอร์** และรีจิสเตอร์นั้นเองที่ **"เก็บที่อยู่จริง" (Effective Address)** ของข้อมูลในหน่วยความจำหลัก
- **การทำงาน:**
    1. CPU ดึงคำสั่งที่มี Register Indirect Operand เข้ามา
    2. CPU ตีความ OpCode และพบว่า Operand Field ชี้ไปยังหมายเลขรีจิสเตอร์ (เช่น R1)
    3. **CPU เข้าถึงรีจิสเตอร์ R1 เพื่อ "อ่านค่าที่อยู่" ที่อยู่ใน R1** (สมมติว่า R1 เก็บค่า 1000h ไว้)
    4. CPU นำค่า Address (1000h) ที่อ่านได้จากรีจิสเตอร์นั้นไปวางบน **Address Bus**
    5. **Memory Unit** อ่านที่อยู่นั้น และส่งข้อมูลจากตำแหน่งนั้นกลับมายัง CPU ผ่าน **Data Bus**



รูปที่ 3-21 การเข้าถึงข้อมูลในหน่วยความจำหลักแบบระบุแอดเดรสแบบรีจิสเตอร์ทางอ้อม (Register Indirect Addressing Mode)

ข้อดี:

- **มีความยืดหยุ่นสูง:** สามารถเปลี่ยนที่อยู่ของข้อมูลที่จะเข้าถึงได้ง่ายๆ โดยการเปลี่ยนค่าในรีจิสเตอร์นั้น (เช่น ใช้ในการวนลูปเข้าถึง Array หรือโครงสร้างข้อมูลต่างๆ)
- **คำสั่งสั้นกว่า Direct:** เพราะ Operand Field ระบุแค่ชื่อรีจิสเตอร์ ซึ่งใช้บิตน้อยกว่าการระบุ Address หน่วยความจำหลักโดยตรง

- **ข้อเสีย:**
  - **ช้ากว่า Direct เล็กน้อย:** เพราะต้องมีการเข้าถึงรีจิสเตอร์ 1 ครั้ง เพื่ออ่านค่า Address ก่อนที่จะนำ Address นั้นไปเข้าถึงหน่วยความจำหลักอีกครั้ง (เพิ่มขั้นตอนไป 1 ขั้น)
- *ตัวอย่างคำสั่ง (Assembler Syntax):*
  - *LOAD R1, [R2] : โหลดข้อมูลจากหน่วยความจำที่ **ที่อยู่ซึ่งเก็บอยู่ในรีจิสเตอร์** R2 มาเก็บในรีจิสเตอร์ R1*
    - *(ถ้า R2 มีค่า 1000h, คำสั่งนี้จะโหลดข้อมูลจาก M[1000h] มา R1)*
  - *STORE [R3], R4 : เก็บข้อมูลจากรีจิสเตอร์ R4 ไปยังหน่วยความจำที่ **ที่อยู่ซึ่งเก็บอยู่ในรีจิสเตอร์** R3*

สรุปความแตกต่างที่สำคัญ:

- *Direct: ที่อยู่ของข้อมูล **อยู่ในคำสั่งโดยตรง***
- *Register Indirect: ที่อยู่ของข้อมูล **อยู่ในรีจิสเตอร์** ที่คำสั่งชี้ไป*

ทั้งสองโหมดนี้ใช้ในการเข้าถึงข้อมูลในหน่วยความจำหลัก แต่มีวิธีการระบุที่อยู่และระดับความยืดหยุ่นที่แตกต่างกัน การเข้าใจโหมดเหล่านี้เป็นสิ่งสำคัญในการวิเคราะห์การทำงานของโปรแกรมในระดับ Assembly Language

16. การทำงานคำสั่ง Indexed Addressing Mode, Relative Addressing Mode และ Stack Addressing Mode

- **Indexed Addressing Mode (การกำหนดแอดเดรสแบบแทนที่/ดัชนี):**
  - **การทำงาน:** ที่อยู่จริงของ Operand ได้จากการนำ "ค่าคงที่ (Offset)" ที่อยู่ในคำสั่ง ไปบวกกับ "ค่าในรีจิสเตอร์ดัชนี (Index Register)"
  - **ตัวอย่าง:** LOAD X, [R1] + Constant (โหลดข้อมูลจากที่อยู่ซึ่งได้จากค่าในรีจิสเตอร์ R1 บวกกับค่าคงที่ มาเก็บในตัวแปร X) มักใช้ในการเข้าถึงข้อมูลในอาเรย์หรือโครงสร้างข้อมูลที่เรียงกัน

- **Relative Addressing Mode (การกำหนดแอดเดรสแบบสัมพันธ์):**
    - **การทำงาน:** ที่อยู่จริงของ Operand ได้จากการนำ "ค่าคงที่ (Offset)" ที่อยู่ในคำสั่ง ไปบวกกับ "ค่าปัจจุบันใน Program Counter (PC)"
    - **ตัวอย่าง:** BRANCH Label ซึ่ง Label อาจถูกแปลงเป็น Offset จาก PC ปัจจุบัน คำสั่งกระโดดเหล่านี้มักใช้ในคำสั่ง Branch เพื่อกระโดดไปยังตำแหน่งที่สัมพันธ์กับคำสั่งปัจจุบัน
- **Stack Addressing Mode (การกำหนดแอดเดรสโดยใช้สแตก):**
    - **การทำงาน:** การเข้าถึงข้อมูลจะทำผ่านโครงสร้างข้อมูลแบบสแตก (Stack) ซึ่งทำงานแบบ Last-In, First-Out (LIFO) โดยมีรีจิสเตอร์ชี้สแตก (Stack Pointer) คอยชี้ไปยังตำแหน่งบนสุดของสแตก
    - **ตัวอย่าง:** คำสั่ง PUSH (ใส่ข้อมูลลงบนสแตก) และ POP (นำข้อมูลออกจากสแตก) การระบุ Operand จะทำโดยปริยายผ่าน Stack Pointer ไม่ได้ระบุที่อยู่โดยตรง
    - **ข้อดี:** เหมาะสำหรับการเรียกใช้ Subroutine และการจัดการพารามิเตอร์

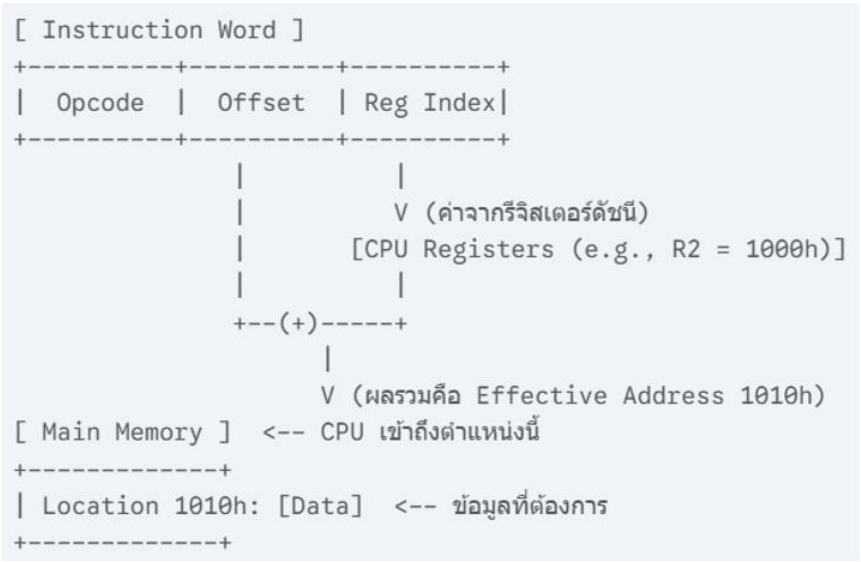
การอธิบายแนวคิดการทำงานของ Indexed, Relative และ Stack Addressing Mode

โหมดเหล่านี้เป็นวิธีการที่ซับซ้อนมากขึ้นในการคำนวณหาที่อยู่จริง (Effective Address) ของข้อมูลในหน่วยความจำ ทำให้โปรแกรมเมอร์มีความยืดหยุ่นในการเข้าถึงข้อมูลมากขึ้น โดยเฉพาะข้อมูลที่จัดเก็บเป็นกลุ่มหรือมีการเปลี่ยนแปลงตำแหน่ง

1. Indexed Addressing Mode (การกำหนดแอดเดรสแบบใช้ดัชนี)

- **แนวคิด:** เปรียบเหมือนกับการมี "บัตรเชิญ" ที่บอกว่า "ให้ไปเริ่มต้นที่เลขที่บ้านในลิ้นชัก A (Index Register) แล้วเดินไปอีก X ก้าว (Offset) นั่นแหละคือข้อมูลที่ต้องการ"
- **คำอธิบาย:** ในโหมดนี้ Effective Address ของ Operand จะได้มาจากการ **นำค่าในรีจิสเตอร์ดัชนี (Index Register) มาบวกกับค่าคงที่ (Constant/Offset)** ที่ระบุในคำสั่ง
    - Effective Address = (Index Register Content) + Offset
- **การทำงาน:**
    1. CPU ดึงคำสั่งที่มี Indexed Operand เข้ามา
    2. CPU ตีความ OpCode และพบว่า Operand Field ระบุชื่อรีจิสเตอร์ดัชนี (เช่น R1) และค่า Offset (เช่น 10)
    3. CPU อ่านค่าจากรีจิสเตอร์ R1 (สมมติ R1 มีค่า 2000h)
    4. **CPU นำค่าใน R1 (2000h) มาบวกกับ Offset (10h) ได้เป็น 2010h** นี่คือ Effective Address

5. CPU นำ Effective Address (2010h) ไปวางบน Address Bus เพื่อเข้าถึงข้อมูลในหน่วยความจำ



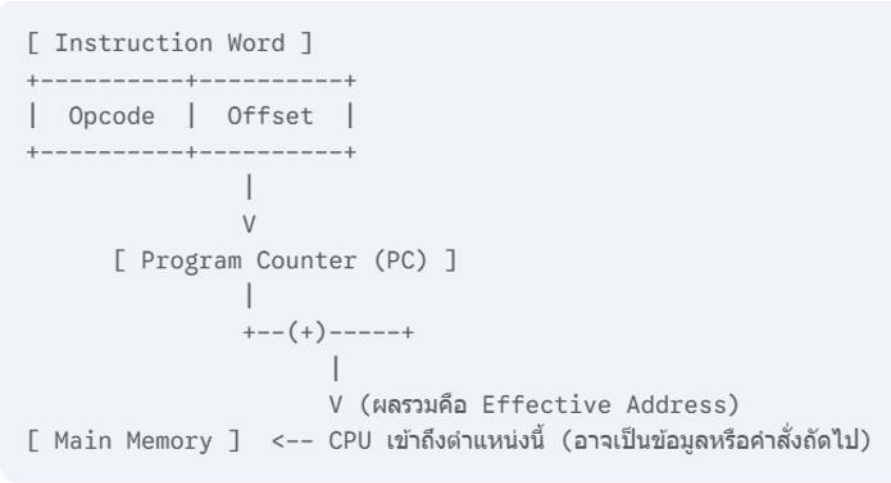
รูปที่ 3-22 แผนผังการไหลของข้อมูลในโหมดการระบุตำแหน่งทางอ้อมผ่านรีจิสเตอร์ (Data Flow in Register Indirect Addressing Mode)

- **ข้อดี:**
  - **เหมาะกับการเข้าถึง Array/Table:** สามารถเข้าถึงสมาชิกแต่ละตัวใน Array ได้อย่างมีประสิทธิภาพ โดยเปลี่ยนแค่ค่าใน Index Register หรือ Offset
  - **มีความยืดหยุ่นสูง:** ใช้ในการวนลูปเพื่อประมวลผลข้อมูลจำนวนมาก หรือเข้าถึงโครงสร้างข้อมูลที่มีการจัดเรียงเป็นแถว
- **ข้อเสีย:**
  - **ช้ากว่า Direct เล็กน้อย:** เพราะต้องมีการบวกเลขเพื่อคำนวณ Effective Address ก่อนเข้าถึงหน่วยความจำ
- ***ตัวอย่างคำสั่ง (Assembler Syntax):***
  - *LOAD R1, 10[R2] (หรือ LOAD R1, [R2+10]) : โหลดข้อมูลจากหน่วยความจำที่* ***(ค่าใน R2 + 10)*** *มาเก็บในรีจิสเตอร์ R1*
    - *(ถ้า R2 มีค่า 1000h, คำสั่งนี้จะโหลดข้อมูลจาก M[1010h] มา R1)*

2. Relative Addressing Mode (การกำหนดแอดเดรสแบบสัมพันธ์)

- **แนวคิด:** เปรียบเหมือนกับการมี "บัตรเชิญ" ที่บอกว่า "ให้ไปจากตำแหน่งปัจจุบันที่เรากำลังยืนอยู่ (Program Counter) แล้วเดินไปอีก X ก้าว (Offset) นั่นแหละคือข้อมูลหรือคำสั่งที่ต้องการ"
- **คำอธิบาย:** ในโหมดนี้ Effective Address ของ Operand หรือคำสั่งที่จะ Jump/Branch ไป จะได้มาจากการ **นำค่าใน Program Counter (PC) มาบวกกับค่าคงที่ (Constant/Offset)** ที่ระบุในคำสั่ง “Effective Address = (Program Counter Content) + Offset

- **การทำงาน:**
    1. CPU ดึงคำสั่งที่มี Relative Operand เข้ามา
    2. CPU ตีความ OpCode และพบว่า Operand Field ระบุค่า Offset
    3. **CPU นำค่าปัจจุบันของ PC มาบวกกับ Offset เพื่อคำนวณ Effective Address**
    4. นำ Effective Address ไปใช้ในการเข้าถึงข้อมูล หรือใช้เป็นที่อยู่ใหม่สำหรับ PC (ในกรณีของ Branch/Jump)

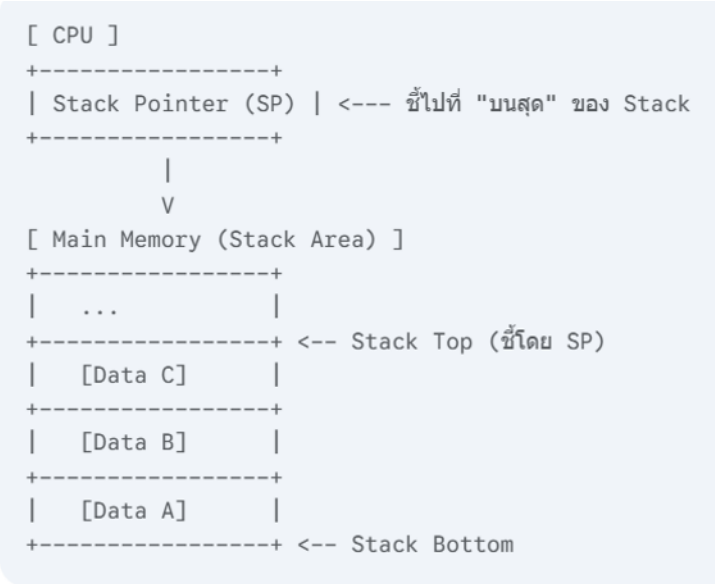


รูปที่ 3-23 กลไกการระบุตำแหน่งแบบการขยับร่วมกับรีจิสเตอร์ (Displacement Addressing Mechanism)

- **ข้อดี:**
    - **Program Relocatable (ย้ายที่ได้):** โปรแกรมที่ใช้ Relative Addressing สามารถโหลดไปทำงานที่ตำแหน่งใดๆ ในหน่วยความจำก็ได้ โดยไม่ต้องแก้ไขโค้ด (เพราะทุกการอ้างอิงเป็นแบบสัมพันธ์กับ PC)
    - **มักใช้กับคำสั่ง Branch/Jump:** ทำให้การกระโดดไปยังตำแหน่งใกล้เคียงใช้คำสั่งสั้นลง
- **ข้อเสีย:**
    - **เข้าถึงได้เฉพาะตำแหน่งใกล้เคียง:** Offset มีขนาดจำกัด ทำให้ไม่สามารถกระโดดไปหรืออ้างอิงข้อมูลที่อยู่ไกลมากๆ ในหน่วยความจำได้
- *ตัวอย่างคำสั่ง (Assembler Syntax):*
    - *BNE LABEL_A (Branch If Not Equal to LABEL_A) : คำสั่งนี้จะคำนวณ Offset โดยอ้างอิงจาก PC ไปยัง LABEL_A*
        - *(ถ้า PC = 1000h, LABEL_A = 1050h, Offset = 50h, คำสั่งคือ BNE +50h)*
    - *LOAD R1, [PC + 20] : โหลดข้อมูลจากหน่วยความจำที่ **(ค่าใน PC + 20)** มาเก็บในรีจิสเตอร์ R1*

3. Stack Addressing Mode (การกำหนดแอดเดรสโดยใช้สแตก)

- **แนวคิด:** เปรียบเหมือนกับการมี "กองจาน" (Stack) โดย "คำสั่ง" จะไม่ระบุว่าต้องไปหยิบจานใบไหน แต่จะรู้โดยปริยายว่าต้องทำงานกับ "จานใบที่อยู่บนสุด" เสมอ และมี "พนักงาน" (Stack Pointer) คอยชี้ที่จานใบนั้น
- **คำอธิบาย:** ในโหมดนี้ **Operand ไม่ได้ถูกระบุที่อยู่โดยตรงในคำสั่ง** แต่จะถูกเข้าถึงโดยอ้างอิงจาก **Stack Pointer (SP)** ซึ่งเป็นรีจิสเตอร์พิเศษที่ชี้ไปยังตำแหน่งบนสุดของ Stack (พื้นที่หน่วยความจำที่จัดสรรไว้สำหรับการทำงานแบบ LIFO)
- **การทำงาน:**
    - **PUSH:** เพิ่มข้อมูลลง Stack (ลดค่า SP แล้วเขียนข้อมูล)
    - **POP:** ดึงข้อมูลออกจาก Stack (อ่านข้อมูลแล้วเพิ่มค่า SP)
    - คำสั่งเช่น PUSH AX (ผลักค่าใน AX ลง Stack), POP BX (ดึงค่าจาก Stack มาใส่ BX) จะไม่มี Operand Field ที่เป็น Address



รูปที่ 3-24 กลไกการระบุตำแหน่งข้อมูลแบบสแตก (Stack Addressing Mode)

- **ข้อดี:**
    - **คำสั่งสั้นมาก:** เพราะไม่จำเป็นต้องระบุที่อยู่ของ Operand เลย
    - **เหมาะสำหรับ Call/Return, การส่ง Parameters:** ใช้ในการจัดการ Subroutine Call และการผ่านค่าพารามิเตอร์ให้กับฟังก์ชัน
    - **จัดการ Stack โดยอัตโนมัติ:** CPU จะดูแล Stack Pointer ให้เอง
- **ข้อเสีย:**
    - **จำกัดการเข้าถึง:** ทำงานได้เฉพาะกับข้อมูลที่อยู่บนสุดของ Stack เท่านั้น
    - **ไม่ยืดหยุ่นในการเข้าถึงข้อมูลแบบสุ่ม:** ไม่สามารถเข้าถึงข้อมูลที่อยู่กลาง Stack ได้โดยตรง

- *ตัวอย่างคำสั่ง (Assembler Syntax):*
    - *PUSH AX : นำค่าในรีจิสเตอร์ AX ไปใส่บน Stack*
    - *POP BX : ดึงค่าบนสุดของ Stack มาใส่ในรีจิสเตอร์ BX*

<u>ชุดข้อสอบเพื่อทบทวนความเข้าใจบทที่ 3</u>: เรื่องชุดคำสั่งและวัฏจักรคำสั่งของ CPU

คำชี้แจง: เลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว

1. ชุดคำสั่ง (Instruction Set) หมายถึงข้อใดที่ถูกต้องและครบถ้วนที่สุด?
   ก. ส่วนของหน่วยความจำหลักสำหรับเก็บโปรแกรมทั้งหมด
   ข. กลุ่มของคำสั่งทั้งหมดที่กำหนดการทำงานให้แก่ CPU
   ค. ระบบการจัดการข้อมูลเข้าและข้อมูลออกของคอมพิวเตอร์
   ง. กลุ่มของรีจิสเตอร์ที่ใช้ในการควบคุมการทำงานของระบบ
   เฉลย: ข.
2. OpCode (Operation Code) มีหน้าที่สำคัญที่สุดในคำสั่งเครื่องอย่างไร?
   ก. ระบุแอดเดรสของคำสั่งถัดไปที่ CPU ต้องดึงมาใช้งาน
   ข. ระบุรหัสที่บอกประเภทของการดำเนินการที่ CPU ต้องปฏิบัติ
   ค. ระบุขนาดและชนิดของข้อมูลที่ใช้ในการประมวลผลคำสั่ง
   ง. จัดเก็บผลลัพธ์ของการประมวลผลข้อมูลสุดท้ายของคำสั่ง
   เฉลย: ข.
3. Operand ในโครงสร้างคำสั่งทำหน้าที่หลักอย่างไร?
   ก. ระบุตำแหน่งของตัวถูกดำเนินการเพื่อใช้ในการประมวลผล
   ข. กำหนดสัญญาณควบคุมสำหรับส่วนควบคุม (Control Unit)
   ค. เป็นรหัสที่บอกว่า CPU ต้องทำการกระโดดหรือไม่กระโดด
   ง. ทำหน้าที่เป็นตัวควบคุมความเร็วของระบบสัญญาณนาฬิกา
   เฉลย: ก.
4. วัฏจักรคำสั่งของ CPU แบ่งออกเป็นสองขั้นตอนหลักตามลำดับอย่างไร?
   ก. Data Operation Cycle และ Store Cycle
   ข. Interrupt Cycle และ Indirection Cycle
   ค. Fetch Cycle (การดึงคำสั่ง) และ Execute Cycle (การปฏิบัติการ)
   ง. Operand Fetch Cycle และ Operand Store Cycle
   เฉลย: ค.
5. กระบวนการ Instruction Address Calculation (IAC) มีบทบาทใด?
   ก. ตีความรหัสคำสั่งที่ดึงเข้ามาใน Instruction Register
   ข. คำนวณหาตำแหน่งที่อยู่ของคำสั่งถัดไปที่ CPU จะต้องดึงมา
   ค. อ่านข้อมูลของตัวถูกดำเนินการจากตำแหน่งที่กำหนดไว้

ง. บันทึกผลลัพธ์ของการประมวลผลลงในหน่วยความจำหลัก

เฉลย: ข.

6. การทำงานหลักในขั้นตอน **Instruction Fetch (IF)** คือข้อใด?

ก. อ่านรหัสคำสั่งจากหน่วยความจำมาไว้ใน Instruction Register (IR)

ข. คำนวณแอดเดรสจริงของ Operand ก่อนการประมวลผลทันที

ค. วิเคราะห์รหัส OpCode เพื่อแยกส่วนของการดำเนินการ

ง. เพิ่มค่าของรีจิสเตอร์ Program Counter (PC) เป็นสองเท่า

เฉลย: ก.

7. **Instruction Operation Decoding (IOD)** มีวัตถุประสงค์เพื่ออะไร?

ก. บันทึกค่ารีจิสเตอร์ PC ลงในหน่วยความจำ Stack

ข. วิเคราะห์คำสั่งเพื่อทราบประเภทของ Operation และ Operand

ค. ดึงข้อมูล Operand จากหน่วยความจำมาไว้ใน CPU

ง. ใช้ในการส่งผลลัพธ์การดำเนินการกลับไปยังหน่วยความจำ

เฉลย: ข.

8. ขั้นตอน **Execute (การปฏิบัติการ)** เป็นการทำงานใด?

ก. CPU ตรวจสอบสถานะของ Interrupt Flag และ Data Flag

ข. CPU กำหนดค่าเริ่มต้นใหม่ให้กับ Accumulator Register

ค. CPU สลับการทำงานจาก User Mode ไปยัง Kernel Mode

ง. CPU สั่งให้ ALU ดำเนินการตาม OpCode ที่ถูกตีความแล้ว

เฉลย: ง.

9. **Operand Address Calculation (OAC)** เป็นการดำเนินการใด?

ก. การตรวจสอบความถูกต้องของรหัส OpCode ที่ถูกดึงมา

ข. การคำนวณตำแหน่งที่อยู่ของคำสั่งถัดไปโดยใช้ PC

ค. การหาตำแหน่งที่อยู่จริงของตัวถูกดำเนินการในหน่วยความจำ

ง. การแปลงข้อมูลจากรูปแบบหนึ่งไปอีกรูปแบบหนึ่งก่อนใช้

เฉลย: ค.

10. กระบวนการ **Operand Fetch (OF)** มีจุดประสงค์เพื่ออะไร?

ก. ดึงข้อมูลจริงของตัวถูกดำเนินการจากหน่วยความจำมาที่ CPU

ข. อ่านรหัสคำสั่งจากหน่วยความจำหลักมาไว้ในรีจิสเตอร์ PC

ค. ส่งสัญญาณควบคุมไปยัง I/O Module เพื่อเริ่มต้นการทำงาน

ง. ทำการบันทึกผลลัพธ์สุดท้ายของการดำเนินการลงในหน่วยความจำ

เฉลย: ก.

11. ส่วนสำคัญที่เข้ามามีบทบาทหลักในขั้นตอน **Data Operation (DO)** คืออะไร?

ก. Memory Buffer Register (MBR)
ข. Program Counter (PC)
ค. Instruction Register (IR)
ง. Arithmetic Logic Unit (ALU)

เฉลย: ง.

12. **Operand Store (OS)** คือขั้นตอนใดในการประมวลผลคำสั่ง?

ก. การบันทึกผลลัพธ์ที่ได้จากการประมวลผลลงในตำแหน่งที่กำหนด
ข. การย้ายข้อมูลจากรีจิสเตอร์หนึ่งไปยังอีกรีจิสเตอร์หนึ่ง
ค. การคำนวณแอดเดรสที่อยู่ของคำสั่งถัดไปในวัฏจักร
ง. การตรวจสอบสถานะของอุปกรณ์ Input/Output

เฉลย: ก.

13. ส่วนประกอบคำสั่ง **Operation Code** มีความจำเป็นอย่างไร?

ก. ใช้ในการอ้างถึงตำแหน่งของข้อมูลทั้งหมดในหน่วยความจำ
ข. เป็นรหัสสำคัญที่ใช้ในการบอกถึงกิจกรรมที่ต้องปฏิบัติ
ค. ใช้ในการควบคุมความเร็วสัญญาณนาฬิกาของ CPU
ง. ใช้ในการระบุว่าต้องมีการอ้างถึง Operand กี่ตัว

เฉลย: ข.

14. ส่วนประกอบ **การอ้างถึงตัวถูกดำเนินการ (Operand Reference)** มีบทบาทอย่างไร?

ก. ระบุขนาดของรหัสคำสั่งในหน่วยความจำหลัก
ข. ระบุตำแหน่งของข้อมูลที่ต้องการนำมาใช้หรือบันทึกผล
ค. ทำหน้าที่ในการควบคุมลำดับของการทำงานแบบ Pipeline
ง. ใช้ในการกำหนดสถาปัตยกรรมของหน่วยความจำแคช

เฉลย: ข.

15. **การอ้างถึงคำสั่งถัดไป** ส่วนใหญ่ถูกกำหนดไว้อย่างไรในคำสั่งส่วนมาก?

ก. ถูกกำหนดโดย Stack Pointer ที่ชี้ไปยังคำสั่งถัดไป
ข. ถูกกำหนดให้เป็นคำสั่งที่อยู่ติดกันในหน่วยความจำ (PC+1)
ค. ถูกระบุโดย Address Field ขนาดใหญ่ในรหัสคำสั่ง
ง. ถูกกำหนดโดย I/O Controller ผ่านทาง Control Bus

เฉลย: ข.

16. การออกแบบชุดคำสั่ง **Operation Repertory** เกี่ยวข้องกับสิ่งใด?

    ก. การกำหนดจำนวนและประเภทของคำสั่งที่ CPU สามารถทำได้

    ข. การออกแบบจำนวนของ Memory Access Cycle ที่อนุญาต

    ค. การกำหนดขนาดของรีจิสเตอร์ที่มองเห็นได้โดยโปรแกรมเมอร์

    ง. การเลือกใช้ระบบ Bus แบบ Address Bus หรือ Data Bus

    เฉลย: ก.

17. การออกแบบชุดคำสั่งในส่วนของ **Data Type** หมายถึงสิ่งใด?

    ก. รูปแบบทางกายภาพของแพ็กเกจ CPU ที่ใช้

    ข. ชนิดของข้อมูลที่ OpCode สามารถจัดการได้ เช่น ตัวเลข ตัวอักษร

    ค. วิธีการจัดการสัญญาณ Interrupt ที่เข้ามาสู่ CPU

    ง. ลำดับการเข้าถึงข้อมูลในหน่วยความจำแคชระดับต่าง ๆ

    เฉลย: ข.

18. **Instruction Format (รูปแบบคำสั่ง)** จะกำหนดอะไรในการออกแบบชุดคำสั่ง?

    ก. ความเร็วสัญญาณนาฬิกาสูงสุดที่ CPU สามารถทำงานได้

    ข. โครงสร้าง ขนาดของฟิลด์ OpCode และ Address ในคำสั่ง

    ค. อัตราส่วนของการทำงานแบบ Pipelining ใน CPU

    ง. วิธีการสร้างวงจรควบคุม (Control Unit) ให้ทำงานอย่างรวดเร็ว

    เฉลย: ข.

19. การออกแบบชุดคำสั่งในส่วนของ **Register** มีความเกี่ยวข้องกับอะไร?

    ก. กำหนดขนาดความจุของหน่วยความจำหลัก (Main Memory)

    ข. กำหนดรีจิสเตอร์ที่โปรแกรมเมอร์มองเห็นและสามารถใช้งานได้

    ค. การกำหนดว่าควรมีหน่วยความจำแคชกี่ระดับในระบบ

    ง. การเลือกใช้เทคโนโลยีการผลิตชิป Microprocessor

    เฉลย: ข.

20. การออกแบบชุดคำสั่งในส่วนของ **Addressing** มีวัตถุประสงค์ใด?

    ก. กำหนดวิธีการที่ Operand ถูกอ้างถึงหรือระบุตำแหน่งที่อยู่

    ข. กำหนดขนาดของคำสั่งเครื่องให้มีขนาดเท่ากันทั้งหมด

    ค. กำหนดรูปแบบของการทำงานแบบ Parallel Processing

    ง. กำหนดลำดับความสำคัญของ Interrupt Signal

    เฉลย: ก.

21. คำสั่งในกลุ่ม **Data Transfer** มีจุดประสงค์หลักเพื่ออะไร?

    ก. ย้ายข้อมูลระหว่างส่วนประกอบต่าง ๆ โดยไม่มีการเปลี่ยนค่า

    ข. เปรียบเทียบค่าข้อมูลสองค่าเพื่อตั้งค่า Flag ของ CPU

    ค. ดำเนินการทางคณิตศาสตร์เช่นการบวกและการลบเท่านั้น

    ง. ควบคุมการทำงานของอุปกรณ์ I/O ที่เชื่อมต่อกับระบบ

    เฉลย: ก.

22. คำสั่ง **ADD** และ **MULTIPLY** จัดอยู่ในประเภทใด?

    ก. Logical Operation

    ข. Data Transfer Operation

    ค. System Control Operation

    ง. Arithmetic Operation

    เฉลย: ง.

23. คำสั่ง **OR** และ **XOR** จัดอยู่ใน Operation ประเภทใด?

    ก. Conversion Operation

    ข. Arithmetic Operation

    ค. Logical Operation

    ง. Input/Output Operation

    เฉลย: ค.

24. คำสั่งประเภท **System Control** มีวัตถุประสงค์เพื่ออะไร?

    ก. ใช้ในการเข้ารหัสข้อมูลก่อนส่งไปยัง I/O Module

    ข. ใช้ในการเปลี่ยนแปลงลำดับการประมวลผลหรือควบคุม CPU

    ค. ใช้ในการจัดการหน่วยความจำเสมือนของระบบปฏิบัติการ

    ง. ใช้สำหรับการอ้างถึงข้อมูลแบบทันที (Immediate Value)

    เฉลย: ข.

25. คำสั่ง **IN** และ **OUT** ใช้ในการสื่อสารกับอุปกรณ์ภายนอก จัดอยู่ในประเภทใด?

    ก. Conversion Operation

    ข. System Control Operation

    ค. Arithmetic Operation

    ง. Input/Output Operation

    เฉลย: ง.

26. คำสั่งที่ใช้ในการ **แปลงข้อมูล** จาก ASCII เป็น Binary จัดอยู่ใน Operation ประเภทใด?

    ก. Logical Operation

    ข. System Control Operation

    ค. Conversion Operation

    ง. Data Transfer Operation

    เฉลย: ค.

27. **Unconditional Branch** คือคำสั่งควบคุมระบบแบบใด?

    ก. กระโดดไปยังแอดเดรสที่ระบุหากรีจิสเตอร์เป็นศูนย์

    ข. กระโดดไปยังแอดเดรสที่กำหนดไว้เสมอโดยไม่มีเงื่อนไข

    ค. ข้ามคำสั่งถัดไปหากเงื่อนไขที่กำหนดเป็นจริง

    ง. หยุดการทำงานของ CPU เพื่อรอสัญญาณ Interrupt

    เฉลย: ข.

28. **Conditional Branch** เป็นการทำงานอย่างไร?

    ก. กระโดดไปยังแอดเดรสที่ระบุโดยไม่ต้องตรวจสอบ Flag

    ข. กระโดดไปยังแอดเดรสที่กำหนดต่อเมื่อเงื่อนไขเป็นจริงเท่านั้น

    ค. ทำการเรียกใช้ Subroutine ทันทีโดยไม่ต้องบันทึก PC

    ง. ทำการย้ายข้อมูลระหว่างรีจิสเตอร์และหน่วยความจำเท่านั้น

    เฉลย: ข.

29. คำสั่ง **Skip** (ข้ามคำสั่งถัดไปหากเงื่อนไขจริง) มีผลต่อ Program Counter (PC) อย่างไร?

    ก. PC ถูกเพิ่มค่าให้ข้ามคำสั่งถัดไป (PC = PC + 2)

    ข. PC ถูกกำหนดค่าใหม่ตามแอดเดรสที่ระบุในคำสั่ง

    ค. PC ถูกบันทึกไว้ใน Stack ก่อนการข้ามคำสั่ง

    ง. PC ถูกกำหนดให้เป็นศูนย์เพื่อหยุดการทำงาน

    เฉลย: ก.

30. คำสั่ง **Procedure Call** มีหน้าที่หลักอย่างไร?

    ก. สั่งให้ CPU หยุดการทำงานชั่วคราวเพื่อประหยัดพลังงาน

    ข. บันทึกตำแหน่งสุดท้ายของโปรแกรมลงในรีจิสเตอร์ IR

    ค. เรียกใช้ Interrupt Service Routine (ISR) ทันที

    ง. บันทึก Return Address ลงใน Stack แล้วกระโดดไปทำ Subroutine

    เฉลย: ง.

31. หน้าที่หลักของ **Subroutine Call** คล้ายกับ Procedure Call อย่างไร?

    ก. Subroutine Call ไม่มีการบันทึก PC ลงใน Stack เลย

    ข. ทั้งสองคำสั่งใช้การบันทึกตำแหน่งกลับเพื่อกระโดดไปทำชุดคำสั่งย่อย

    ค. Subroutine Call ใช้ได้เฉพาะกับคำสั่งแบบ 1-Address เท่านั้น

    ง. Procedure Call ใช้สำหรับโปรแกรมระดับ Kernel เท่านั้น

    เฉลย: ข.

32. คำสั่งควบคุมอุปกรณ์ **Input/Output** มีบทบาทใดที่สำคัญ?

ก. ใช้ในการเข้ารหัสข้อมูลที่รับมาจาก I/O Module

ข. ใช้ในการคำนวณแอดเดรสของ I/O Port โดยใช้ PC

ค. ใช้ในการจัดลำดับความสำคัญของ Interrupt ที่เข้ามา

ง. ใช้ในการเริ่มต้น หยุด หรือตรวจสอบสถานะของอุปกรณ์ I/O

เฉลย: ง.

33. ในคำสั่งแบบ **4-Address Instruction** Address ตำแหน่งที่สี่ใช้ทำอะไร?

ก. ระบุตำแหน่งที่อยู่ของคำสั่งถัดไปที่จะถูกดึงมาประมวลผล

ข. ระบุตำแหน่งของรีจิสเตอร์ Stack Pointer ที่ใช้งาน

ค. ระบุขนาดของข้อมูล Operand ที่จะถูกนำมาใช้

ง. ระบุประเภทของ Control Signal ที่ต้องการส่งออกไป

เฉลย: ก.

34. คำสั่งแบบ **3-Address Instruction** ในรูปแบบ **ADD A, B, C** หมายถึงอะไร?

ก. นำค่าใน B บวกกับ C แล้วเก็บผลลัพธ์ไว้ในตำแหน่ง A

ข. นำค่าใน A บวกกับ B แล้วเก็บผลลัพธ์ไว้ในตำแหน่ง C

ค. นำค่าคงที่ C บวกกับ A แล้วเก็บในรีจิสเตอร์ B

ง. โหลดค่า A, B, C มาไว้ในรีจิสเตอร์ CPU พร้อมกัน

เฉลย: ก.

35. ข้อเสียหลักของ **2-Address Instruction** เมื่อเทียบกับ 3-Address คืออะไร?

ก. คำสั่งมีความซับซ้อนในการตีความสูงมากเกินไป

ข. ต้องใช้คำสั่งย้ายข้อมูล (MOVE) เพิ่มเพื่อรักษาค่าเริ่มต้น

ค. ไม่สามารถใช้กับ Logical Operation ที่ซับซ้อนได้

ง. ใช้พื้นที่หน่วยความจำในการจัดเก็บข้อมูลน้อยเกินไป

เฉลย: ข.

36. ในคำสั่งแบบ **1-Address Instruction** Operand ที่สองถูกระบุโดยปริยายว่าเป็นอะไร?

ก. หน่วยความจำหลัก (Main Memory)

ข. Stack Pointer Register

ค. Accumulator Register (รีจิสเตอร์ตัวสะสม)

ง. Instruction Register

เฉลย: ค.

37. **Immediate Addressing Mode** แอดเดรสฟิลด์ (Address Field) ของคำสั่งบรรจุอะไร?

    ก. ที่อยู่ของรีจิสเตอร์ที่เก็บข้อมูลจริง

    ข. ที่อยู่ของหน่วยความจำที่เก็บข้อมูล

    ค. ข้อมูลจริง (ค่าคงที่) ที่จะนำมาใช้ในการดำเนินการ

    ง. ที่อยู่ของคำสั่งถัดไปที่จะต้องกระโดดไป

    เฉลย: ค.

38. ใน **Register Addressing Mode** แอดเดรสฟิลด์ของคำสั่งระบุอะไร?

    ก. หมายเลขของรีจิสเตอร์ที่เก็บข้อมูลที่จะนำมาใช้

    ข. ตำแหน่งที่อยู่จริงในหน่วยความจำหลัก

    ค. ค่าคงที่ที่ต้องนำมาบวกกับ PC

    ง. ที่อยู่ของ Input/Output Port

    เฉลย: ก.

39. ข้อดีที่สำคัญที่สุดของการใช้ **Register Addressing Mode** คืออะไร?

    ก. สามารถอ้างถึงหน่วยความจำหลักได้ทั้งหมดอย่างง่ายดาย

    ข. เข้าถึงข้อมูลได้รวดเร็วที่สุดและใช้ Address Field สั้น

    ค. เหมาะสำหรับคำสั่งที่ต้องทำงานกับข้อมูลหลายไบต์

    ง. เป็นโหมดการอ้างแอดเดรสที่มีความยืดหยุ่นสูงมาก

    เฉลย: ข.

40. ลักษณะเด่นของ **Direct Addressing Mode** คือข้อใด?

    ก. แอดเดรสฟิลด์ระบุตำแหน่งที่อยู่จริงของ Operand ในหน่วยความจำ

    ข. ต้องมีการเข้าถึงหน่วยความจำสองครั้งก่อนได้ข้อมูล

    ค. ข้อมูลจริงที่ต้องการถูกเก็บอยู่ในรหัสคำสั่งเลย

    ง. แอดเดรสถูกคำนวณจากค่าใน PC บวกกับค่าคงที่

    เฉลย: ก.

41. **Register Indirect Addressing Mode** แอดเดรสฟิลด์ในคำสั่งทำหน้าที่อะไร?

    ก. ระบุหมายเลขรีจิสเตอร์ที่เก็บตำแหน่งที่อยู่จริงของ Operand

    ข. บรรจุข้อมูลจริง (Immediate Value) ที่จะนำมาใช้ทันที

    ค. ชี้ไปยังตำแหน่งของ Stack ที่เก็บ Return Address

    ง. ระบุ Offset ที่ต้องนำมาบวกกับ Base Register

    เฉลย: ก.

42. ข้อแตกต่างหลักระหว่าง Direct และ Register Indirect คืออะไร?

    ก. Direct ใช้ได้กับหน่วยความจำเท่านั้น ส่วน Indirect ใช้กับ Register

    ข. Direct ชี้ไปที่ข้อมูลโดยตรง ส่วน Indirect ชี้ไปที่รีจิสเตอร์ที่เก็บแอดเดรส

    ค. Register Indirect ต้องเข้าถึงหน่วยความจำเพียง 1 ครั้งเท่านั้น

    ง. Direct มีความเร็วในการทำงานสูงกว่า Register Indirect เสมอ

    เฉลย: ข.

43. การคำนวณแอดเดรสจริงใน **Indexed Addressing Mode** เป็นอย่างไร?

    ก. แอดเดรสจริง = (ค่าใน Register + ค่าคงที่ในคำสั่ง)

    ข. แอดเดรสจริง = (ค่าใน PC + ค่าคงที่ในคำสั่ง)

    ค. แอดเดรสจริง = (ค่าใน Stack Pointer)

    ง. แอดเดรสจริง = (ค่าคงที่ในคำสั่ง)

    เฉลย: ก.

44. **Indexed Addressing Mode** มักใช้ในกรณีใด?

    ก. การเข้าถึงค่าคงที่หรือตัวแปรเดี่ยวที่ไม่เปลี่ยนแปลง

    ข. การเข้าถึงสมาชิกของโครงสร้างข้อมูลแบบแถวลำดับ (Array)

    ค. การจัดการกับ Stack ในการเรียกใช้ Subroutine

    ง. การกระโดดแบบมีเงื่อนไขภายใน Subroutine

    เฉลย: ข.

45. **Relative Addressing Mode** คำนวณแอดเดรสจริงอย่างไร?

    ก. แอดเดรสจริง = (ค่าใน Instruction Register + ค่าคงที่)

    ข. แอดเดรสจริง = (ค่าใน Stack Pointer + ค่าคงที่)

    ค. แอดเดรสจริง = (ค่าใน Accumulator + ค่าคงที่)

    ง. แอดเดรสจริง = (ค่าใน Program Counter + ค่าคงที่)

    เฉลย: ง.

46. ข้อดีของการใช้ **Relative Addressing Mode** คืออะไร?

    ก. สามารถอ้างถึงหน่วยความจำขนาดใหญ่ได้อย่างไร้ขีดจำกัด

    ข. ช่วยให้โปรแกรมสามารถทำงานได้แม้ถูกย้ายตำแหน่ง (Relocatable)

    ค. ทำให้การเรียกใช้ I/O Port มีความรวดเร็วและปลอดภัยขึ้น

    ง. เป็นโหมดเดียวที่ใช้ในการกระโดดแบบมีเงื่อนไขในโปรแกรม

    เฉลย: ข.

47. ใน **Stack Addressing Mode** คำสั่ง **PUSH** มีผลต่อ Stack Pointer (SP) อย่างไร?

    ก. SP ถูกปรับค่าให้ชี้ไปยังตำแหน่งใหม่ก่อนการบันทึกข้อมูล

    ข. SP ถูกกำหนดค่าใหม่จาก Operand ที่ระบุในคำสั่ง

    ค. SP ถูกเพิ่มค่าขึ้นโดยไม่คำนึงถึงขนาดของข้อมูล

    ง. SP ไม่เปลี่ยนแปลงเลยในระหว่างการทำ PUSH

    เฉลย: ก.

48. เหตุผลที่ Stack Addressing Mode ไม่ต้องมี Address Field ใน Operand คืออะไร?

    ก. ข้อมูล Operand ถูกเก็บในรีจิสเตอร์ของ CPU เสมอ

    ข. คำสั่ง PUSH/POP ทำงานกับหน่วยความจำหลักโดยตรง

    ค. Operand ถูกกำหนดโดยปริยายว่าเป็นข้อมูลที่อยู่บนสุดของ Stack

    ง. Stack Pointer ทำหน้าที่เป็น Address Field ที่มองไม่เห็น

    เฉลย: ค.

49. เมื่อเกิด **Interrupt** ขึ้นระหว่างประมวลผลคำสั่ง CPU จะตอบสนองในวงรอบใด?

    ก. Fetch Cycle ข. Execute Cycle

    ค. Interrupt Cycle ง. Data Operation Cycle

    เฉลย: ค.

50. คำสั่ง **ADD R1, #50** ในที่นี้ Operand ที่สอง (#50) ใช้ Addressing Mode ใด?

    ก. Register Addressing Mode

    ข. Direct Addressing Mode

    ค. Immediate Addressing Mode

    ง. Relative Addressing Mode

    เฉลย: ค.